# Fixed-Width vs. Multiprecision Digit Generation for JSON Canonicalization: A Four-Algorithm Empirical Study

Mark Lenhardt

Lattice Substrate

**Abstract**

Benchmarks that measure only speed are insufficient when the code under test must satisfy a formatting specification: a fast but non-conformant implementation is unusable. We introduce *conformance-gated benchmarking*, a methodology that treats specification correctness as a binary prerequisite for performance measurement. The methodology gates every candidate on an oracle vector suite, documents noise floors and minimum detectable effect sizes for every comparison, and controls false-discovery rate across the full comparison matrix. It is general, applicable wherever correctness is a hard prerequisite, and was developed from the practical discovery that existing JSON Canonicalization Scheme (RFC 8785) implementations delegate number formatting to language runtimes that silently violate the required ECMA-262 `Number::toString` policy.

We apply the methodology to a case study comparing four digit-generation algorithms (three fixed-width: Schubfach, Ryū, Dragonbox; one multiprecision: Burger–Dybvig) inside an RFC 8785 canonicalization pipeline, with cross-language validation in Go and Rust (8 implementations, 1,644 statistical comparisons). Conformance is verified against 286,362 oracle vectors generated from V8 and validated against SpiderMonkey and JavaScriptCore. All three fixed-width algorithms perform within 1–8% of each other across both languages, while each is 1.4–1.8× faster than Burger–Dybvig on number-dense workloads ($\nu(D) \geq 40\%$) in Go API benchmarks; these pipeline-level ratios are consistent with larger isolation-level speedups attenuated by Amdahl's Law, since digit generation accounts for approximately 15–40% of total pipeline cost; the advantage replicates at 2.3–3.0× in Rust (excluding two pure-numeric workloads producing 8–9× ratios discussed in §8). On string-dominant and deeply-nested inputs the difference narrows to <5%. All eight implementations produce byte-identical output; Go binaries are additionally verified under cross-compilation for x86-64 and arm64. We release the benchmark infrastructure, oracle vectors, and all raw data as a reproducibility artifact.

**Keywords:** JSON canonicalization, floating-point formatting, Schubfach, Ryū, Dragonbox, Burger–Dybvig, RFC 8785, ECMA-262, benchmark methodology, cross-language validation

# 1 Introduction

Specification-constrained formatting is common in systems where serialized bytes are hashed, signed, or used as consensus inputs: a single divergent byte invalidates every downstream hash and signature. Yet the standard benchmarking approach for formatting code, which measures speed in isolation and assumes correctness, is inadequate here. A fast implementation that silently deviates from the specification is unusable, because the deviation may not surface until production signatures fail to verify. This paper introduces *conformance-gated benchmarking*, a methodology that treats specification correctness as a binary prerequisite: candidates must pass an oracle vector suite before any timing data is collected. The methodology documents noise floors and minimum detectable effect sizes for every comparison, applies Benjamini–Hochberg FDR correction across the full comparison matrix, and is general, applicable to any domain where correctness is a hard prerequisite for meaningful performance measurement.

The methodology was developed from an engineering problem, not adopted as an academic principle. Building a correct JSON Canonicalization Scheme (RFC 8785) [18] primitive required implementing ECMA-262 `Number::toString` [5] number formatting from scratch, because existing RFC 8785 implementations delegate to language runtimes (Go's `strconv.FormatFloat`, for example) that do not implement the ECMA-262 formatting policy (they make different notation choices at key boundaries: `1e20` vs. the required `100000000000000000000`; `1e-06` vs. the required `0.000001`) and whose underlying algorithms have changed across versions. Conformance verification against 286,362 oracle vectors generated from V8 and independently validated against SpiderMonkey and JavaScriptCore confirmed these gaps. A correct Burger–Dybvig [3] implementation with exact multiprecision arithmetic was built, validated against the full vector suite, and deployed; profiling then revealed its heap allocation as a bottleneck on number-dense workloads. Replacing Burger–Dybvig with the fixed-width Schubfach [9] algorithm eliminated digit-generation allocation entirely while producing byte-identical output. The natural follow-up question, *is Schubfach specifically the right choice, or would Ryū [1] or Dragonbox [12] perform equivalently?*, asked with statistical rigor inside a pipeline context, is the case study of this paper.

Several deployment contexts place canonicalization on the critical path. In high-frequency signature verification, every document must be canonicalized before its signature can be checked, making formatting throughput a direct bottleneck. Content-addressed storage systems hash the canonical form at both ingestion and retrieval, so canonicalization cost scales linearly with object throughput. Distributed consensus protocols require byte-identical serialization for state-transition agreement, invoking canonicalization on every proposal and vote. In each scenario, a constant-factor improvement in the formatting stage translates directly to throughput gains: the 1.4–1.8× speedup we measure on number-dense payloads corresponds to freeing approximately 24% of a CPU core at 1,000 canonicalizations per second on a representative 14 KB workload.

Beyond throughput, the choice of digit-generation algorithm is a correctness concern in systems where canonicalized bytes are hashed and signed at every layer of a processing pipeline. In deployment contexts such as deterministic infrastructure, where data-processing steps execute in sandboxed isolation and every governed artifact is schema-validated, canonically serialized, and cryptographically bound, a formatting divergence at the number-

rendering layer does not produce one wrong document: it can invalidate hashes computed over those bytes, signatures computed over those hashes, and audit trails that chain those signatures. The digit-generation algorithm sits at the foundation of a determinism guarantee that higher layers cannot independently recover. This makes both the correctness validation (conformance gating) and the performance characterization reported here directly relevant to infrastructure where canonicalization is not optional but load-bearing [18].

This paper makes three contributions:

1. A conformance-gated benchmark methodology, developed from the practical experience that delegating to standard library formatters produces output that passes casual testing but fails specification-level verification. The methodology gates performance measurement on correctness: implementations must pass 286,362 oracle vectors and 776 structural conformance cases before timing data is collected. We document noise floors and minimum observable effect sizes for all comparisons. The methodology is general and reusable for any specification-constrained formatting evaluation.

2. To our knowledge, the first empirical comparison of four digit-generation algorithms (three fixed-width: Schubfach, Ryū, Dragonbox; one multiprecision: Burger–Dybvig) under ECMA-262/RFC 8785 constraints inside a canonicalization pipeline, showing that all three fixed-width algorithms perform within 1–8% of each other while each achieves 1.4–1.8× speedup over Burger–Dybvig on number-dense workloads ($\nu(D) \geq 40\%$) in Go API benchmarks; the advantage replicates at 2.3–3.0× in Rust. The fixed-width equivalence is confirmatory (consistent with Adams and Champagne Gareau & Lemire), but the pipeline context and Amdahl's Law characterization showing how isolation-level speedups attenuate at the system level are new. String-dominant and deeply-nested inputs show <5% difference.

3. A cross-language validation design (4×2 matrix of algorithm × language) that separates algorithmic effects from language/runtime effects, with Go cross-compilation determinism verified for x86-64 and arm64 via QEMU user-mode emulation. All evidence is released as a reproducibility artifact.

# 2 Background

## 2.1 RFC 8785: JSON Canonicalization Scheme

RFC 8785 [18] defines a deterministic JSON serialization by specifying: (1) lexicographic key ordering using UTF-16 code unit comparison, (2) no insignificant whitespace, and (3) floating-point formatting matching ECMA-262 `Number::toString`.

The floating-point requirement is technically demanding. It requires producing the *shortest decimal representation* that round-trips to the original IEEE 754 binary64 value under roundTiesToEven, then applying the ECMA-262 formatting policy (exponential notation thresholds, sign handling, integer elision rules).

## 2.2 ECMA-262 Number Formatting

ECMA-262 [5] §6.1.6.1.20 specifies `Number::toString` as a two-stage process (formerly §7.1.12.1 in older editions):

1. **Digit generation**: find the shortest decimal significand $d$ and exponent $n$ such that $d \times 10^{n-k} = x$ (where $k$ is the number of digits in $d$) and the value round-trips.

2. **Formatting**: apply branch logic based on the relationship between $k$ (digit count) and $n$ (decimal exponent position) to produce the final string. Key thresholds: $n \geq 1 \wedge n \leq 21$ produces fixed notation; $n \leq 0$ or $n > 21$ produces exponential notation.

## 2.3 IEEE 754 and Shortest Representation

IEEE 754-2019 [11] binary64 has a 52-bit significand, 11-bit exponent, and 1 sign bit. The *shortest representation* property requires finding the decimal string with fewest digits that uniquely identifies the original binary64 value among all representable values. This is the core algorithmic challenge addressed by both Burger–Dybvig and Schubfach.

# 3 Related Work

Extensive prior work has optimized float-to-string conversion in isolation. To our knowledge, this paper is the first to evaluate these algorithms inside a specification-constrained canonicalization pipeline where conformance is a binary gate. We summarize the most relevant prior work, organized by the problem dimension each addresses.

## 3.1 Multiprecision Foundations

Steele and White [19] established the foundational framework for accurate float-to-string conversion. Burger and Dybvig [3] extended this to shortest-representation printing using arbitrary-precision rational arithmetic; their algorithm is our multiprecision baseline. Gay's `dtoa.c` [7] provides the foundational practical implementation and remains widely used in language runtimes as a reference or fallback. All three works evaluate algorithms in isolation and do not consider performance inside a pipeline where formatting policy is externally constrained.

## 3.2 Fixed-Width Approaches

Loitsch [15] demonstrated that integer-only arithmetic suffices for the common case with Grisu. However, Grisu3 falls back to arbitrary precision for ∼0.5% of inputs, making conformance guarantees harder to establish in specification-constrained contexts.

Adams [1] proposed Ryū, achieving full coverage with fixed-width 128-bit arithmetic and lookup tables. Adams provides the most comprehensive prior performance comparison of fixed-width vs. multiprecision approaches (we refer the reader to Adams's literature review

rather than replicating it), but evaluates algorithms in isolation on microbenchmarks without specification constraints.

Giulietti [9] introduced the Schubfach algorithm, which we implement and evaluate within the RFC 8785 pipeline. Jeon [12] extends and refines the Schubfach approach with Dragonbox, improving table compression and branch structure. Go 1.26 adopted Dragonbox for `strconv` [16], providing additional motivation for evaluating fixed-width approaches in specification-constrained contexts.

## 3.3 Comprehensive Experimental Review

Champagne Gareau and Lemire [4] provide a comprehensive experimental comparison of modern float-to-string algorithms, evaluating Dragon4, Grisu3, Ryū, Schubfach, Dragonbox, and `std::to_chars` in C++. Their work evaluates algorithms in isolation on microbenchmarks. The present study complements theirs by embedding the same algorithms inside a specification-constrained canonicalization pipeline where (a) ECMA-262 formatting policy is a hard constraint, (b) correctness against 286,362 oracle vectors is a binary gate, and (c) digit generation is only one component of the total cost.

The complementary direction, fast string-to-float parsing, is addressed by Lemire [14], whose Eisel-Lemire algorithm achieves gigabyte-per-second parsing throughput. While outside our scope, this work shares the fixed-width arithmetic philosophy that motivates Schubfach and Dragonbox.

## 3.4 Deterministic Serialization Standards

Beyond JSON, deterministic encoding is required by other serialization formats. CBOR (RFC 8949) [2] specifies deterministic encoding rules (§4.2) for binary data, addressing analogous concerns about canonical representation. W3C RDF Dataset Canonicalization [20] addresses graph-level canonicalization (structural equivalence of RDF datasets), a structurally different problem from the byte-level serialization determinism required by RFC 8785. Our conformance-gated methodology could be adapted to other deterministic serialization benchmarks.

## 3.5 JSON Canonicalization

RFC 8785 [18] is the primary standard. Prior to RFC 8785, ad-hoc canonicalization schemes existed in various ecosystems (e.g., JSON-LD canonicalization, OLPC canonical JSON). None specify the floating-point formatting requirement with the precision of ECMA-262 [5].

Prior to this work, conformance verification against ECMA-262 oracle vectors revealed that existing RFC 8785 implementations couple number formatting to language runtime internals without implementing the ECMA-262 formatting policy directly. The standard pattern (delegating to the runtime's float-to-string function and post-processing the result) is fragile: Go's `strconv.FormatFloat`, the dependency in a widely used Go RFC 8785 implementation, has undergone three algorithm changes (Grisu3 through Go 1.13, Ryū from Go 1.14, Dragonbox from Go 1.26) and had a confirmed rounding bug (Go issue #29491, filed by the RFC 8785 author, where `FormatFloat` returned wrong digits for specific values

on certain platforms). This ecosystem gap motivated both the custom implementation and the conformance-gated methodology presented here.

### 3.6 Benchmark Methodology

Our conformance-before-performance approach draws on the principle that correctness is a prerequisite for meaningful performance comparison. Georges et al. [8] established rigorous statistical methods for Java performance evaluation, advocating confidence intervals and effect sizes over single-point estimates. Mytkowicz et al. [17] demonstrated that measurement bias (link order, environment size) can reverse performance conclusions, motivating our randomized task ordering and environment documentation. Kalibera and Jones [13] formalized the analysis of multi-level experimental designs in benchmarking. Our use of permutation tests and BCa bootstrap confidence intervals follows this modern statistical practice for performance evaluation.

## 4 Algorithms

We evaluate only full-coverage algorithms that produce shortest-representation output for all IEEE 754 binary64 inputs without fallback to arbitrary precision. This excludes Grisu3 [15], which falls back to multiprecision arithmetic for approximately 0.5% of inputs, confounding the algorithmic comparison in a specification-constrained context where every input must be handled by the same code path.

### 4.1 Definitions

**Definition 1** (Minimal Significand)**.** Given a finite IEEE 754 binary64 value $x$, the *minimal significand* is the shortest decimal significand $d$ (with digit count $k$) such that parsing $d \times 10^{n-k}$ under roundTiesToEven recovers $x$ exactly.

**Definition 2** (Minimal Printed String)**.** The *minimal printed string* of $x$ is the output of ECMA-262 `Number::toString` applied to $x$: the minimal significand formatted according to the ECMA-262 branch table.

*Remark* 1. Definitions 1 and 2 are not equivalent in string length. Consider $x = 10^{20}$: the minimal significand is $d = 1$ ($k = 1$, $n = 21$). Since $n = 21 \leq 21$, ECMA-262 uses fixed notation, yielding `100000000000000000000` (21 characters). Now consider $x = 10^{21}$: again $d = 1$ ($k = 1$, $n = 22$), but $n > 21$ triggers exponential notation, yielding `1e+21` (5 characters). The $n = 21$ boundary creates divergence between significand minimality and printed-string length. This means implementations cannot optimize for shorter output strings because the ECMA-262 formatting is spec-mandated.

### 4.2 Burger–Dybvig (Multiprecision)

The Burger–Dybvig algorithm [3] uses arbitrary-precision rational arithmetic to compute the shortest decimal representation. It maintains big-integer numerator/denominator pairs

and iteratively extracts decimal digits. The algorithm is general (handles any radix) but requires heap-allocated multiprecision integers, with cost proportional to the magnitude of the exponent.

In our Go implementation, this uses `math/big.Int` operations. The `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers across calls.

### 4.3 Schubfach (Fixed-Width)

The Schubfach algorithm [9] uses fixed-width 64/128-bit arithmetic, relying on `math/bits` for 128-bit multiplication (`Mul64`, `Add64`) and bit inspection (`TrailingZeros64`). It uses precomputed tables of powers of ten and 128-bit multiplications to determine the shortest decimal significand. No heap allocation is required in the digit-generation path.

The key advantage is constant-time (with respect to exponent magnitude) operation and zero GC pressure. The precomputed table is approximately 5 KiB.

### 4.4 Ryū (Fixed-Width)

The Ryū algorithm [1] was the first to achieve full coverage shortest-output generation using only fixed-width 128-bit arithmetic and precomputed lookup tables. Like Schubfach, it computes a rounding interval around the input value and converts it to decimal via table-based 128-bit multiplication. Ryū uses a `lastRemovedDigit` heuristic for digit trimming: digits are removed one at a time from both ends of the interval, tracking the last removed digit for round-to-even tie-breaking. Our implementation uses the same interval computation as Schubfach (symmetric for regular values, asymmetric at power-of-two boundaries); Adams's original formulation uses a slightly different interval setup, but the shortest-output guarantee and round-to-even tie-breaking are preserved.

### 4.5 Dragonbox (Fixed-Width)

The Dragonbox algorithm [12] extends the fixed-width approach with two key optimizations: a *shorter interval* case for power-of-two boundaries (where the lower half of the rounding interval is narrower), and *two-digit-at-a-time* trimming (dividing by 100 instead of 10 in the digit extraction loop). Our implementation separates the shorter-interval and general cases into distinct functions, preserving the algorithm's structural innovation.

### 4.6 ECMA-262 Formatting Stage

All four algorithms share an identical formatting stage that converts the minimal significand $(d, k, n)$ triple into the final ECMA-262-compliant string. This stage implements the branch table from ECMA-262 §6.1.6.1.20 and is pure string manipulation with no arithmetic.

## 5 Implementation

We maintain eight implementations with a common CLI ABI:

- `impl-schubfach` (Go): Schubfach number formatting.
- `impl-json-canon` (Go): Burger–Dybvig number formatting.
- `impl-ryu` (Go): Ryū number formatting.
- `impl-dragonbox` (Go): Dragonbox number formatting.
- `impl-schubfach-rs` (Rust): Schubfach-aligned Rust path.
- `impl-json-canon-rs` (Rust): Burger–Dybvig-aligned Rust path.
- `impl-ryu-rs` (Rust): Ryū-aligned Rust path.
- `impl-dragonbox-rs` (Rust): Dragonbox-aligned Rust path.

All eight implementations contain their own digit-generation code and do not delegate to the standard library's float-to-string routines (e.g., Go's `strconv.FormatFloat` or Rust's `std::fmt`). This is necessary because the standard routines do not implement ECMA-262 formatting policy. Consequently, Go 1.26's adoption of Dragonbox for `strconv` [16] does not affect our measurements.

Within each language, all four implementations share identical JSON parser, serializer, error taxonomy, and ECMA-262 formatting code; only the `jcsfloat` digit-generation module differs.[1] This isolation ensures that performance differences are attributable to the digit-generation algorithm alone.

The lab harness treats each implementation as a black-box canonicalizer with identical CLI contracts (`canonicalize`, `verify`, exit-code policy, and error-class expectations). This preserves algorithm comparability while allowing language/runtime differences to be measured independently.

Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`) to align optimization policy with production benchmarking practice.

## 5.1 Rust Implementation Notes

All four Rust implementations use native `u128` arithmetic for the 128-bit intermediate products required by the fixed-width algorithms, eliminating the need for compiler intrinsics or multi-word emulation. The Rust Burger–Dybvig implementation uses the `num-bigint` crate (v0.4), which provides Karatsuba multiplication for large operands, comparable in optimization level to Go's `math/big.Int` with its assembly-accelerated x86-64 inner loops. Buffer lifecycle is managed by Rust's RAII ownership model (deterministic `Drop`), removing the need for explicit pooling strategies such as Go's `sync.Pool`.

All Rust implementations share an identical JSON parser and serializer; only the `jcsfloat` digit-generation module differs between them, mirroring the Go design. Rust binaries are

[1] A `c0`-tracking correction in the Dragonbox two-digit trimming loop (`removed / 10` → `removed % 10`) was identified during a pre-submission audit and corrected in both the Go and Rust implementations. A related variable-ordering fix was applied to the three Rust fixed-width implementations. Both corrections are correctness-neutral for IEEE 754 inputs (all 286,362 oracle vectors pass identically before and after the fix) but restore logical consistency with the algorithm specification.

approximately 424–456 KB (stripped), compared to 2.5–2.7 MB for the statically-linked Go binaries.

### 5.2 Platform Independence and Fused-Multiply-Add

IEEE 754 compliance guarantees that each floating-point result is correctly rounded to destination precision, but it does not guarantee that the same source expression produces identical bits on all platforms. Fused-multiply-add (FMA) instructions compute $a \times b + c$ with a single rounding rather than two, producing results that can differ by one ULP from the unfused sequence. Go's compiler permits FMA for any multiply-add expression [10]; the opt-out is an explicit `float64()` cast forcing intermediate rounding. A real-world consequence was observed in December 2025, when a Go database (DoltHub) produced different query plans on ARM vs. x86 because the compiler emitted FMA on ARM but not x86, causing a one-ULP difference that flipped a cost comparison [6].

For digit-generation algorithms used in canonicalization, where the formatted bytes are cryptographically bound, platform-dependent output is unacceptable. The Burger–Dybvig implementation with `math/big.Int` is immune by construction: all operations execute on the CPU's integer ALU, where floating-point fusion is inapplicable by type. The three fixed-width algorithms (Schubfach, Ryū, Dragonbox) use `math/bits.Mul64` and `u128` arithmetic for their core computations; we verified that no floating-point expressions in the digit-generation path are fusible. All eight implementations therefore satisfy the integer-arithmetic-only property required for cross-platform determinism.

## 6 Methodology

### 6.1 Conformance-Before-Performance

Our methodology enforces a strict ordering: conformance verification must pass before any performance data is collected. This approach was developed from practical experience: the standard pattern for RFC 8785 number formatting (delegating to a language runtime's float-to-string function) produces output that passes casual testing but fails ECMA-262 boundary-value verification (e.g., notation-switch thresholds at $10^{21}$ and $10^{-7}$, even-digit tie-breaking under Note 2). An implementation that looks fast in a benchmark while producing wrong output is unusable in a specification-constrained context where the formatted bytes are cryptographically bound. The citations to Georges et al. [8] and Mytkowicz et al. [17] reinforce the approach; the engineering experience originated it.

**Conformance gate:** 97 structural test cases per implementation (776 total across 8 implementations) covering key ordering, Unicode handling, numeric formatting, escape sequences, depth limits, and invalid input rejection. All implementations must produce byte-identical output (verified via SHA-256) and identical error classification.

**Oracle vectors:** 286,362 IEEE 754 binary64 values with expected `String(x)` output generated by the V8 JavaScript engine (Node.js v23.3.0, V8 12.9.202.28) and independently validated against SpiderMonkey (JavaScript-C115.10.0) and JavaScriptCore (WebKitGTK 2.50.4). All three engines produce identical output for every vector, confirming cross-engine

agreement on ECMA-262 `Number::toString` semantics. (2,153 values appear in both vector files as overlapping boundary coverage; the total counts file lines, with 284,209 unique values.) Coverage includes all subnormals with bit patterns $\leq 2^{12}$ (4,096 of approximately $4.5 \times 10^{15}$ positive subnormals; these provide dense coverage near zero where notation-switch boundaries and leading-zero sequences create formatting edge cases), subnormal/normal boundary ($\pm 128$ around the transition), all powers of two in the binary64 exponent range (binade boundaries where the digit-generation algorithm must activate asymmetric interval margins), RFC 8785 Appendix B examples with $\pm 2$ neighbors (catching off-by-one errors in boundary comparisons against the specification's own test values), and pseudorandom fill.

## 6.2 Benchmark Design

**CLI benchmarks:** end-to-end process execution measuring wall time, CPU time (user + system), and peak RSS. 15 repetitions per implementation/mode/workload combination, with 3 warmup runs discarded. Task order is randomized with a fixed seed for reproducibility.

**Cross-language comparison pairs:** each of the four algorithms is implemented in Go and Rust, yielding eight implementations. These are compared via 16 pairwise tests:

- 6 within-Go algorithm pairs (e.g., go:schubfach-vs-ryu, go:dragonbox-vs-bd),
- 6 within-Rust algorithm pairs (e.g., rs:schubfach-vs-ryu, rs:dragonbox-vs-bd),
- 4 cross-language same-algorithm pairs (e.g., schubfach:go-vs-rs, ryu:go-vs-rs).

The within-language pairs test the algorithmic effect while controlling for language/runtime; the cross-language pairs test language/runtime effects while controlling for algorithm. Directional consistency across within-language pairs in both Go and Rust provides evidence that algorithmic differences are not solely attributable to language-specific runtime characteristics, though this is not a formal interaction test (see §8). The Rust Burger–Dybvig implementation uses the `num-bigint` crate (v0.4, Karatsuba multiplication for large operands, heap-allocated with RAII lifecycle), providing a production-grade multiprecision library comparable to the Go `math/big`-based path. Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`) to align optimization policy with production benchmarking practice.

**API benchmarks:** Go `testing.B` microbenchmarks (Go implementations) and a custom Rust benchmark harness (`src/bin/bench.rs` in each Rust implementation) measuring canonicalization and verify paths directly, 10 iterations per benchmark. Go's `testing.B` auto-calibrates the number of inner iterations per sample (typically hundreds to millions), whereas the Rust harness times each function call individually ($N$=1) using `std::time::Instant`; both report ns/op. The Rust harness includes 3 warmup iterations (discarded) before measurement to stabilize caches and lazy-initialized data structures (e.g., the `OnceLock`-based power-of-ten cache in the Burger–Dybvig path). The harness uses a counting global allocator (`std::alloc::GlobalAlloc` with atomic counters) to report per-operation allocation bytes and counts, and emits Go-compatible `Benchmark... ns/op` lines so the same statistical pipeline can be applied uniformly. Low coefficients of variation (<5%) are achieved despite $N$=1 because the workload payloads are large enough (0.1–14 KB) that per-call latency is well above timer resolution.

**Measurement granularity note:** The Go and Rust API harnesses differ in measurement granularity. Go's `testing.B` averages over many inner iterations per sample, producing low-variance samples. The Rust harness reports single-call latency, producing higher-variance samples. Within-language comparisons are unaffected since both implementations in a pair use the same harness. Cross-language API comparisons should be interpreted with this asymmetry in mind; we report them for completeness but emphasize within-language results for algorithmic conclusions.

**Differential fuzzing:** 2,000 randomly-generated JSON inputs processed by all implementations, verifying byte-identical output.

## 6.3 Workload Characterization

For a JSON document $D$, we define $\nu(D)$ as the fraction of JSON values that are numbers. Every JSON value is counted once: primitives (strings, numbers, booleans, null) and containers (arrays, objects); container elements are counted separately. For example, `[1, "x", [2, 3]]` contains six values (two arrays, one string, three numbers), so $\nu = 3/6 = 0.50$.

Table 1 reports the structural properties of each benchmark workload, including $\nu(D)$, total value count, maximum nesting depth, and unique key count.

## 6.4 Statistical Protocol

All comparisons use:

- Permutation tests (5,000 resamples) for significance at $\alpha = 0.05$.
- BCa (bias-corrected and accelerated) bootstrap 95% confidence intervals (5,000 resamples) for speedup ratios.
- Cohen's $d$ for effect size.[2]
- Noise floor documentation: standard deviation of the faster implementation ($\sigma = \text{CV} \times \bar{x}_{\text{faster}}$), quantifying measurement noise in the same units as the metric.
- Minimum detectable effect (MDE): $\frac{2.8\sqrt{2} \times \text{CV} \times 100}{\sqrt{N}}$, where $2.8 \approx z_{\alpha/2} + z_\beta$ for $\alpha = 0.05$ and 80% power (approximate MDE for a two-sample $z$-test with equal group sizes; uses only the faster implementation's CV and $n$ as an approximation). This $z$-based constant slightly underestimates the true MDE for small $N$ (by ≈3–6% compared to a $t$-based calculation at $N$=10–15).

[2]Computed as $(\bar{x}_A - \bar{x}_B)/s_p$ where $A$ and $B$ are the first and second implementation in each pair; negative values indicate implementation $A$ is faster. In API benchmarks where coefficient of variation is typically <2%, Cohen's $d$ values can exceed 50 in absolute value; such magnitudes reflect low measurement variance rather than practically meaningful effect-size distinctions. Readers should use the confidence intervals on speedup ratios as the primary measure of practical effect size; Cohen's $d$ is most informative in CLI benchmarks where higher variance produces values in the conventional interpretive range.

Table 1: Workload structural characterization.

| Workload | Bytes | Total Values | Numbers | $\nu(D)$ (%) | Max Depth | Keys |
|---|---|---|---|---|---|---|
| array-2048 | 98,901 | 10,243 | 4,096 | 40.0 | 3 | 8,194 |
| array-256 | 11,914 | 1,283 | 512 | 39.9 | 3 | 1,026 |
| canonical-minimal | 7 | 2 | 1 | 50.0 | 1 | 1 |
| control-escapes | 59 | 4 | 0 | 0.0 | 1 | 3 |
| deep | 1,158 | 141 | 0 | 0.0 | 140 | 140 |
| deep-64 | 510 | 65 | 0 | 0.0 | 64 | 64 |
| escaped-key-order | 18 | 3 | 2 | 66.7 | 1 | 2 |
| large | 463,537 | 40,963 | 16,384 | 40.0 | 3 | 32,770 |
| long-string | 16,403 | 3 | 0 | 0.0 | 1 | 2 |
| medium | 13,740 | 1,283 | 512 | 39.9 | 3 | 1,026 |
| mixed-prod | 5,653 | 458 | 128 | 27.9 | 4 | 393 |
| nested-mixed | 49 | 10 | 4 | 40.0 | 3 | 7 |
| number-heavy | 140 | 13 | 10 | 76.9 | 2 | 4 |
| numeric-boundary | 98 | 9 | 7 | 77.8 | 2 | 2 |
| prod-cloudevents | 1,520 | 82 | 39 | 47.6 | 4 | 76 |
| prod-geojson | 7,006 | 678 | 378 | 55.8 | 7 | 186 |
| prod-k8s-pod | 3,046 | 190 | 25 | 13.2 | 6 | 156 |
| prod-openapi | 4,128 | 270 | 25 | 9.3 | 12 | 233 |
| prod-package-meta | 1,863 | 83 | 12 | 14.5 | 2 | 73 |
| rfc-key-sorting | 202 | 8 | 0 | 0.0 | 1 | 7 |
| small | 45 | 5 | 2 | 40.0 | 1 | 4 |
| surrogate-pair | 29 | 3 | 0 | 0.0 | 1 | 2 |
| unicode | 98 | 4 | 0 | 0.0 | 1 | 3 |
| verify-whitespace | 24 | 5 | 3 | 60.0 | 2 | 2 |

Total Values counts containers (arrays, objects) and their elements separately.
$\nu(D)$ = number values / total values $\times$ 100.

- Benjamini-Hochberg false discovery rate (FDR) control at $q = 0.05$ across all 1,644 comparisons. BH requires independence or positive regression dependency (PRDS); comparisons on the same workload across different pairs share one common implementation (e.g., go:schubfach-vs-bd and go:ryu-vs-bd both include the BD implementation), inducing positive correlation and satisfying PRDS. We report both raw and BH-adjusted $p$-values.

- No outlier exclusion. Permutation tests are nonparametric and robust to non-normality; BCa bootstrap accounts for skewness via the acceleration constant.

- Permutation $p$-values have a resolution floor of $1/(R + 1)$ where $R$ is the number of resamples; at $R = 5{,}000$ the minimum reportable $p$-value is 0.0002. Values reported at this floor should be interpreted as $p < 0.0002$.

Statistical resampling uses deterministic per-comparison seeds derived via FNV-1a hashing of the comparison key, ensuring reproducible $p$-values and confidence intervals across runs.

### 6.5 Environment

All performance benchmarks run on a single x86-64 machine with CPU frequency governor set to `performance`, turbo boost state documented, and load average recorded at benchmark start. Full hardware specification (CPU model, cache hierarchy, RAM, relevant CPU feature flags) is captured automatically by the benchmark harness.

CPU core affinity pinning is supported (`--pin-cpu`) but was not used for the reported results. The i7-12700K has heterogeneous P-core/E-core topology; without pinning, the OS scheduler may migrate processes. This adds variance but does not bias the comparison since both implementations share the same scheduling environment.

Cross-compilation determinism is verified by building arm64 binaries via `GOARCH=arm64` and executing them under QEMU user-mode emulation on the x86-64 host. QEMU user-mode translates system calls but executes floating-point operations on the host CPU; this tests Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 performance benchmarks are deferred to future work on bare-metal hardware.

## 7 Results

### 7.1 Conformance

All eight implementations (Go and Rust × Schubfach, Ryū, Dragonbox, and Burger–Dybvig) pass the full conformance matrix: 776 total cases (97 structural tests × 8 implementations, 0 failures). Output SHA-256 digests are byte-identical across all implementations for valid inputs.

## 7.2 API Benchmarks

API benchmarks isolate the algorithmic cost of the canonicalization function, removing process startup and I/O overhead. These measurements most directly reflect the digit-generation performance difference.

Table 2 summarizes the key API results for the within-language algorithm comparisons (canonicalize mode). Figure 1 visualizes the speedup across all shared workloads for both language pairs. Full results for all pairs, modes, and workloads appear in Appendix A.

Table 2: API benchmark summary: fixed-width vs. Burger–Dybvig (canonicalize mode, mean of $n$=10; Schubfach shown as representative fixed-width algorithm).

| Pair | Workload | Fixed-width (ns/op) | Burger–Dybvig (ns/op) | Speedup |
|---|---|---|---|---|
| go:schubfach-vs-bd | array-2048 | 2546877 | 4136393 | 1.62× |
| go:schubfach-vs-bd | array-256 | 303668 | 475124 | 1.56× |
| go:schubfach-vs-bd | control-escapes | 1302 | 1303 | 1.00× |
| go:schubfach-vs-bd | deep | 36774 | 36553 | 1.01× |
| go:schubfach-vs-bd | deep-64 | 16312 | 16125 | 1.01× |
| go:schubfach-vs-bd | large | 11878849 | 20927988 | 1.76× |
| go:schubfach-vs-bd | long-string | 76513 | 75390 | 1.01× |
| go:schubfach-vs-bd | medium | 339221 | 587210 | 1.73× |
| go:schubfach-vs-bd | mixed-prod | 135323 | 180091 | 1.33× |
| go:schubfach-vs-bd | number-heavy | 13778 | 21535 | 1.56× |
| go:schubfach-vs-bd | numeric-boundary | 12653 | 18967 | 1.50× |
| go:schubfach-vs-bd | rfc-key-sorting | 2981 | 3004 | 1.01× |
| go:schubfach-vs-bd | small | 1236 | 1671 | 1.35× |
| go:schubfach-vs-bd | unicode | 1560 | 1546 | 1.01× |
| go:schubfach-vs-bd | verify-whitespace | 910 | 1558 | 1.71× |
| rs:schubfach-vs-bd | array-2048 | 1516464 | 3710560 | 2.45× |
| rs:schubfach-vs-bd | array-256 | 174300 | 413556 | 2.37× |
| rs:schubfach-vs-bd | control-escapes | 621 | 641 | 1.03× |
| rs:schubfach-vs-bd | deep | 30537 | 29207 | 1.05× |
| rs:schubfach-vs-bd | deep-64 | 11425 | 10906 | 1.05× |
| rs:schubfach-vs-bd | large | 5807225 | 21141686 | 3.64× |
| rs:schubfach-vs-bd | long-string | 17318 | 18083 | 1.04× |
| rs:schubfach-vs-bd | medium | 178012 | 576989 | 3.24× |
| rs:schubfach-vs-bd | mixed-prod | 68972 | 139944 | 2.03× |
| rs:schubfach-vs-bd | number-heavy | 1457 | 13291 | 9.12× |
| rs:schubfach-vs-bd | numeric-boundary | 988 | 10249 | 10.37× |
| rs:schubfach-vs-bd | rfc-key-sorting | 1355 | 1373 | 1.01× |
| rs:schubfach-vs-bd | small | 670 | 1216 | 1.82× |
| rs:schubfach-vs-bd | unicode | 1550 | 1538 | 1.01× |
| rs:schubfach-vs-bd | verify-whitespace | 400 | 1268 | 3.17× |

Full results for all pairs, modes, and workloads in Appendix A.

Table 3 presents the within-language API comparisons with BH-corrected $p$-values, confidence intervals, and effect sizes.

Table 3: Within-language API comparisons (canonicalize mode, representative workloads; BH-significant marked with *). Complete results for all 19 workloads appear in the supplementary materials.

| Pair | Workload | Winner | Speedup | $CI_{95}$ | $p_{adj}$ | $d$ | MDE (%) |
|---|---|---|---|---|---|---|---|
| go:schubfach-vs-bd | array-2048 | schubfach* | 1.62× | [1.612, 1.635] | 0.0004 | -58.02 | 1.3 |
| go:schubfach-vs-ryu | array-2048 | ryu* | 1.01× | [1.006, 1.019] | 0.0039 | 1.48 | 0.5 |
| go:schubfach-vs-dragonbox | array-2048 | dragonbox | 1.00× | [0.997, 1.012] | 0.4479 | 0.43 | 0.8 |
| go:ryu-vs-dragonbox | array-2048 | ryu* | 1.01× | [1.003, 1.013] | 0.0078 | -1.46 | 0.5 |
| go:ryu-vs-bd | array-2048 | ryu* | 1.64× | [1.636, 1.652] | 0.0004 | -74.65 | 0.5 |
| go:dragonbox-vs-bd | array-2048 | dragonbox* | 1.63× | [1.622, 1.640] | 0.0004 | -68.81 | 0.8 |
| rs:schubfach-vs-bd | array-2048 | schubfach-rs* | 2.45× | [2.135, 2.636] | 0.0004 | -12.25 | 20.9 |
| rs:schubfach-vs-ryu | array-2048 | ryu-rs | 1.11× | [1.028, 1.270] | 0.1150 | 0.83 | 5.7 |
| rs:schubfach-vs-dragonbox | array-2048 | dragonbox-rs* | 1.16× | [1.072, 1.319] | 0.0004 | 1.14 | 0.8 |
| rs:ryu-vs-dragonbox | array-2048 | dragonbox-rs* | 1.04× | [1.021, 1.087] | 0.0004 | 1.16 | 0.8 |
| rs:ryu-vs-bd | array-2048 | ryu-rs* | 2.72× | [2.603, 2.769] | 0.0004 | -52.17 | 5.7 |
| rs:dragonbox-vs-bd | array-2048 | dragonbox-rs* | 2.83× | [2.812, 2.837] | 0.0004 | -221.93 | 0.8 |
| go:schubfach-vs-bd | medium | schubfach* | 1.73× | [1.721, 1.740] | 0.0004 | -77.25 | 0.7 |
| go:schubfach-vs-ryu | medium | ryu | 1.00× | [0.996, 1.006] | 0.7398 | 0.20 | 0.7 |
| go:schubfach-vs-dragonbox | medium | schubfach | 1.00× | [0.997, 1.007] | 0.6656 | -0.26 | 0.7 |
| go:ryu-vs-dragonbox | medium | ryu | 1.00× | [0.998, 1.008] | 0.4227 | -0.44 | 0.7 |
| go:ryu-vs-bd | medium | ryu* | 1.73× | [1.723, 1.741] | 0.0004 | -78.38 | 0.7 |
| go:dragonbox-vs-bd | medium | dragonbox* | 1.73× | [1.717, 1.737] | 0.0004 | -74.27 | 0.9 |
| rs:schubfach-vs-bd | medium | schubfach-rs* | 3.24× | [3.214, 3.277] | 0.0004 | -77.40 | 1.3 |
| rs:schubfach-vs-ryu | medium | ryu-rs | 1.01× | [0.976, 1.024] | 0.5391 | 0.41 | 4.3 |
| rs:schubfach-vs-dragonbox | medium | dragonbox-rs* | 1.01× | [1.004, 1.020] | 0.0134 | 1.25 | 0.9 |
| rs:ryu-vs-dragonbox | medium | dragonbox-rs | 1.00× | [0.988, 1.036] | 0.9952 | 0.05 | 0.9 |
| rs:ryu-vs-bd | medium | ryu-rs* | 3.27× | [3.159, 3.321] | 0.0004 | -61.10 | 4.3 |
| rs:dragonbox-vs-bd | medium | dragonbox-rs* | 3.28× | [3.256, 3.314] | 0.0004 | -79.18 | 0.9 |
| go:schubfach-vs-bd | large | schubfach* | 1.76× | [1.751, 1.774] | 0.0004 | -73.08 | 1.1 |
| go:schubfach-vs-ryu | large | ryu | 1.01× | [0.999, 1.012] | 0.1396 | 0.80 | 0.9 |
| go:schubfach-vs-dragonbox | large | dragonbox | 1.00× | [0.995, 1.007] | 0.7341 | 0.21 | 0.7 |
| go:ryu-vs-dragonbox | large | ryu | 1.00× | [1.000, 1.010] | 0.1711 | -0.74 | 0.9 |
| go:ryu-vs-bd | large | ryu* | 1.77× | [1.762, 1.783] | 0.0004 | -77.49 | 0.9 |
| go:dragonbox-vs-bd | large | dragonbox* | 1.76× | [1.754, 1.773] | 0.0004 | -81.52 | 0.7 |
| rs:schubfach-vs-bd | large | schubfach-rs* | 3.64× | [3.545, 3.710] | 0.0004 | -61.47 | 4.4 |
| rs:schubfach-vs-ryu | large | ryu-rs | 1.01× | [0.994, 1.038] | 0.4957 | 0.43 | 0.5 |
| rs:schubfach-vs-dragonbox | large | dragonbox-rs | 1.01× | [0.984, 1.038] | 0.5739 | 0.33 | 3.6 |
| rs:ryu-vs-dragonbox | large | ryu-rs | 1.00× | [0.989, 1.028] | 0.9930 | -0.02 | 0.5 |
| rs:ryu-vs-bd | large | ryu-rs* | 3.68× | [3.644, 3.707] | 0.0004 | -75.77 | 0.5 |
| rs:dragonbox-vs-bd | large | dragonbox-rs* | 3.68× | [3.582, 3.728] | 0.0004 | -66.07 | 3.6 |
| go:schubfach-vs-bd | prod-cloudevents | schubfach* | 1.35× | [1.338, 1.354] | 0.0004 | -42.94 | 1.1 |
| go:schubfach-vs-ryu | prod-cloudevents | schubfach | 1.00× | [0.998, 1.011] | 0.2524 | -0.61 | 1.1 |
| go:schubfach-vs-dragonbox | prod-cloudevents | dragonbox* | 1.01× | [1.009, 1.021] | 0.0008 | 1.97 | 0.7 |
| go:ryu-vs-dragonbox | prod-cloudevents | dragonbox* | 1.02× | [1.014, 1.025] | 0.0004 | 3.03 | 0.7 |
| go:ryu-vs-bd | prod-cloudevents | ryu* | 1.34× | [1.333, 1.347] | 0.0004 | -46.81 | 0.9 |
| go:dragonbox-vs-bd | prod-cloudevents | dragonbox* | 1.37× | [1.360, 1.372] | 0.0004 | -56.40 | 0.7 |
| rs:schubfach-vs-bd | prod-cloudevents | schubfach-rs* | 2.12× | [2.078, 2.153] | 0.0004 | -40.01 | 3.5 |
| rs:schubfach-vs-ryu | prod-cloudevents | ryu-rs* | 1.04× | [1.013, 1.061] | 0.0183 | 1.29 | 3.8 |
| rs:schubfach-vs-dragonbox | prod-cloudevents | dragonbox-rs | 1.03× | [1.001, 1.053] | 0.0964 | 0.88 | 4.2 |
| rs:ryu-vs-dragonbox | prod-cloudevents | ryu-rs | 1.01× | [0.984, 1.037] | 0.5607 | -0.33 | 3.8 |
| rs:ryu-vs-bd | prod-cloudevents | ryu-rs* | 2.20× | [2.146, 2.234] | 0.0004 | -40.07 | 3.8 |
| rs:dragonbox-vs-bd | prod-cloudevents | dragonbox-rs* | 2.17× | [2.125, 2.216] | 0.0004 | -37.50 | 4.2 |
| go:schubfach-vs-bd | deep | json-canon | 1.01× | [1.000, 1.013] | 0.1426 | 0.80 | 0.8 |
| go:schubfach-vs-ryu | deep | schubfach* | 1.01× | [1.001, 1.015] | 0.0445 | -1.10 | 1.1 |
| go:schubfach-vs-dragonbox | deep | schubfach* | 1.01× | [1.007, 1.022] | 0.0035 | -1.69 | 1.1 |
| go:ryu-vs-dragonbox | deep | ryu | 1.01× | [1.000, 1.013] | 0.1833 | -0.70 | 0.9 |
| go:ryu-vs-bd | deep | json-canon* | 1.01× | [1.009, 1.021] | 0.0016 | 2.11 | 0.8 |

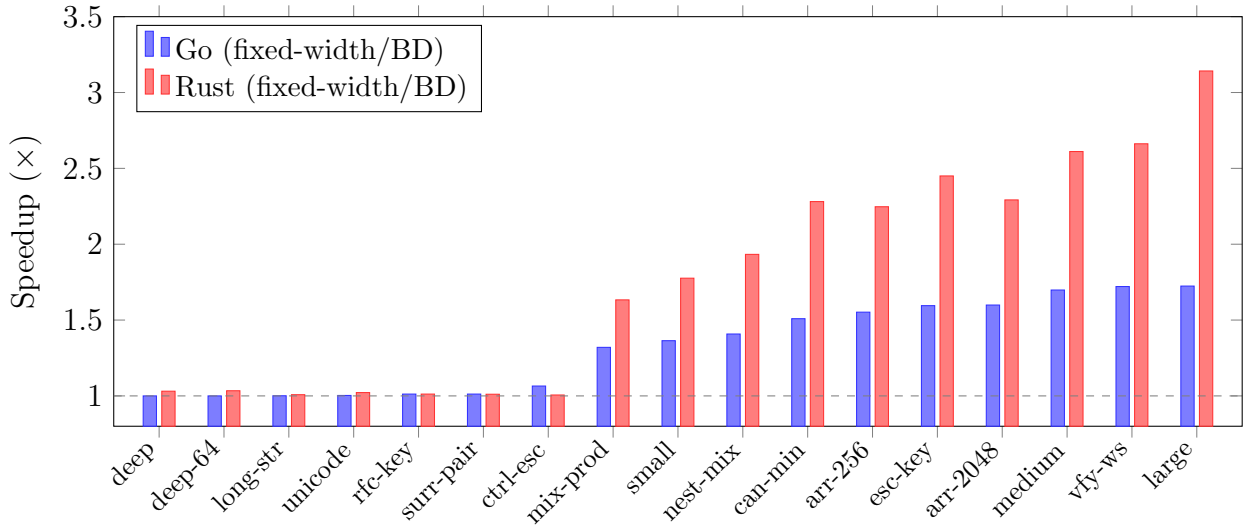


Figure 1: API benchmark speedup of fixed-width over Burger–Dybvig by workload (canonicalize mode, Schubfach shown as representative; Ryū and Dragonbox produce equivalent ratios per §7). Workloads sorted by Go speedup. Values above the dashed line indicate the fixed-width algorithm is faster. Two pure-numeric workloads (`numeric-boundary` and `number-heavy`) are presented separately in §8 due to their degenerate number density ($\nu = 100\%$). Number-dense workloads (right side) show the largest advantage in both languages, consistent with a shared algorithmic cause rather than language-specific runtime effects.

## 7.3 Fixed-Width Algorithm Equivalence

A central finding is that the three fixed-width algorithms (Schubfach, Ryū, Dragonbox) perform within 1–8% of each other across both languages and all workloads. In Go API benchmarks, within-fixed-width comparisons (go:schubfach-vs-ryu, go:schubfach-vs-dragonbox, go:ryu-vs-dragonbox) show speedups of $\leq 1.02\times$ with few BH-significant differences. In Rust, the same pattern holds, with occasional small differences ($\leq 1.13\times$) likely reflecting compilation artifacts rather than algorithmic cost.

This equivalence is consistent with the conclusion that the dominant performance axis is *fixed-width vs. multiprecision*, not the choice among fixed-width algorithms.

## 7.4 CLI Benchmarks

CLI benchmarks capture end-to-end pipeline cost including process startup, argument parsing, file I/O, and output serialization. Estimated process startup overhead (approximately 1.3 ms for Go, 0.5 ms for Rust) dominates for smaller workloads, attenuating the algorithmic signal observed in API benchmarks.

Table 4 summarizes the key CLI results. Figure 2 shows the end-to-end speedup with BH-significance markers. Full results appear in Appendix A.

Table 4: CLI benchmark summary: within-language comparisons (canonicalize mode, $n$=15, 3 warmup discarded).

| Pair | Workload | Impl A (ms) | Impl B (ms) | Speedup | Winner |
|---|---|---|---|---|---|
| go:schubfach-vs-bd | array-2048 | 5.164 | 7.655 | 1.48× | schubfach |
| go:schubfach-vs-bd | array-256 | 1.985 | 2.456 | 1.24× | schubfach |
| go:schubfach-vs-bd | canonical-minimal | 1.473 | 1.676 | 1.14× | schubfach |
| go:schubfach-vs-bd | control-escapes | 1.445 | 1.674 | 1.16× | schubfach |
| go:schubfach-vs-bd | deep | 1.745 | 1.752 | 1.00× | schubfach |
| go:schubfach-vs-bd | deep-64 | 1.484 | 1.610 | 1.08× | schubfach |
| go:schubfach-vs-bd | escaped-key-order | 1.386 | 1.541 | 1.11× | schubfach |
| go:schubfach-vs-bd | large | 16.256 | 25.085 | 1.54× | schubfach |
| go:schubfach-vs-bd | long-string | 1.715 | 1.948 | 1.14× | schubfach |
| go:schubfach-vs-bd | medium | 2.066 | 2.403 | 1.16× | schubfach |
| go:schubfach-vs-bd | mixed-prod | 1.747 | 2.075 | 1.19× | schubfach |
| go:schubfach-vs-bd | nested-mixed | 1.492 | 1.538 | 1.03× | schubfach |
| go:schubfach-vs-bd | number-heavy | 1.447 | 1.742 | 1.20× | schubfach |
| go:schubfach-vs-bd | numeric-boundary | 1.542 | 1.793 | 1.16× | schubfach |
| go:schubfach-vs-bd | prod-cloudevents | 1.587 | 1.764 | 1.11× | schubfach |
| go:schubfach-vs-bd | prod-geojson | 1.878 | 2.317 | 1.23× | schubfach |
| go:schubfach-vs-bd | prod-k8s-pod | 1.694 | 1.871 | 1.10× | schubfach |
| go:schubfach-vs-bd | prod-openapi | 1.468 | 1.865 | 1.27× | schubfach |
| go:schubfach-vs-bd | prod-package-meta | 1.710 | 1.694 | 1.01× | json-canon |
| go:schubfach-vs-bd | rfc-key-sorting | 1.519 | 1.623 | 1.07× | schubfach |
| go:schubfach-vs-bd | small | 1.361 | 1.643 | 1.21× | schubfach |
| go:schubfach-vs-bd | surrogate-pair | 1.481 | 1.508 | 1.02× | schubfach |
| go:schubfach-vs-bd | unicode | 1.490 | 1.540 | 1.03× | schubfach |
| go:schubfach-vs-bd | verify-whitespace | 1.517 | 1.637 | 1.08× | schubfach |
| rs:schubfach-vs-bd | array-2048 | 2.506 | 4.304 | 1.72× | schubfach-rs |
| rs:schubfach-vs-bd | array-256 | 0.797 | 1.151 | 1.44× | schubfach-rs |
| rs:schubfach-vs-bd | canonical-minimal | 0.565 | 0.630 | 1.11× | schubfach-rs |
| rs:schubfach-vs-bd | control-escapes | 0.555 | 0.601 | 1.08× | schubfach-rs |
| rs:schubfach-vs-bd | deep | 0.652 | 0.694 | 1.06× | schubfach-rs |
| rs:schubfach-vs-bd | deep-64 | 0.579 | 0.628 | 1.09× | schubfach-rs |
| rs:schubfach-vs-bd | escaped-key-order | 0.550 | 0.653 | 1.19× | schubfach-rs |
| rs:schubfach-vs-bd | large | 7.680 | 16.824 | 2.19× | schubfach-rs |
| rs:schubfach-vs-bd | long-string | 0.637 | 0.695 | 1.09× | schubfach-rs |
| rs:schubfach-vs-bd | medium | 0.858 | 1.312 | 1.53× | schubfach-rs |
| rs:schubfach-vs-bd | mixed-prod | 0.690 | 0.920 | 1.33× | schubfach-rs |
| rs:schubfach-vs-bd | nested-mixed | 0.555 | 0.690 | 1.24× | schubfach-rs |
| rs:schubfach-vs-bd | number-heavy | 0.558 | 0.713 | 1.28× | schubfach-rs |
| rs:schubfach-vs-bd | numeric-boundary | 0.559 | 0.706 | 1.26× | schubfach-rs |
| rs:schubfach-vs-bd | prod-cloudevents | 0.597 | 0.771 | 1.29× | schubfach-rs |
| rs:schubfach-vs-bd | prod-geojson | 0.698 | 1.158 | 1.66× | schubfach-rs |
| rs:schubfach-vs-bd | prod-k8s-pod | 0.644 | 0.754 | 1.17× | schubfach-rs |
| rs:schubfach-vs-bd | prod-openapi | 0.616 | 0.855 | 1.39× | schubfach-rs |
| rs:schubfach-vs-bd | prod-package-meta | 0.596 | 0.723 | 1.21× | schubfach-rs |
| rs:schubfach-vs-bd | rfc-key-sorting | 0.572 | 0.627 | 1.10× | schubfach-rs |
| rs:schubfach-vs-bd | small | 0.547 | 0.743 | 1.36× | schubfach-rs |
| rs:schubfach-vs-bd | surrogate-pair | 0.546 | 0.616 | 1.13× | schubfach-rs |
| rs:schubfach-vs-bd | unicode | 0.560 | 0.623 | 1.11× | schubfach-rs |
| rs:schubfach-vs-bd | verify-whitespace | 0.571 | 0.694 | 1.21× | schubfach-rs |

Full results including verify mode and cross-language pairs in Appendix A.

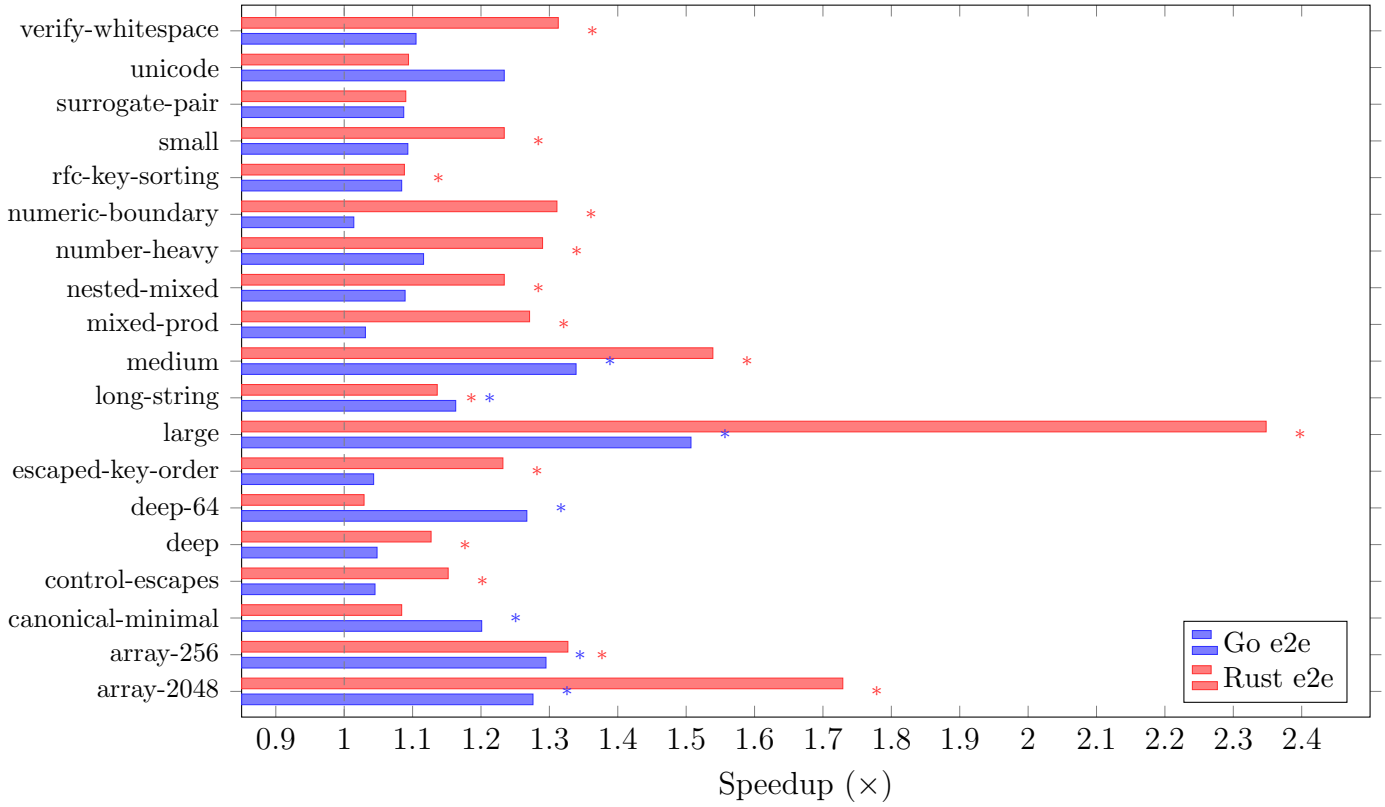


Figure 2: CLI end-to-end speedup of fixed-width over Burger–Dybvig by workload (canonicalize mode, 15 repetitions each; Schubfach shown as representative). Asterisks (∗) mark BH-significant comparisons ($q < 0.05$). The Rust within-language pair achieves significance on 15 of 19 workloads; the Go pair on 7 of 19 workloads (canonicalize mode). Process startup overhead (≈1.3 ms Go, ≈0.5 ms Rust) attenuates the algorithmic signal more strongly in Go.

### 7.5 Statistical Comparisons

Within-language CLI comparisons (canonicalize mode) with BH-corrected $p$-values, confidence intervals, and effect sizes are reported in Table 7 (Appendix A). The patterns mirror the API results: fixed-width algorithms outperform Burger–Dybvig on number-dense workloads, with smaller effect sizes due to process startup dilution. The full 1,644-comparison statistical report appears in the supplementary materials.

### 7.6 Cross-Language Validation

All eight implementations pass the full conformance matrix (776 total cases: 97 structural tests $\times$ 8 implementations, 0 failures) and produce byte-identical output on all oracle vectors. 2,000 differential fuzz cases yield zero cross-implementation divergences.

The statistical report includes 16 comparison classes organized into three groups:

- 6 within-Go algorithm pairs and 6 within-Rust algorithm pairs (testing algorithmic effects within each language),
- 4 cross-language same-algorithm pairs (testing language/runtime effects while controlling for algorithm).

In CLI end-to-end measurement, Rust implementations are approximately 2–3$\times$ faster than their Go counterparts, primarily due to lower process startup overhead (Rust native binary vs. Go runtime initialization). In API benchmarks, the within-Rust algorithmic ordering replicates the within-Go ordering: all three fixed-width algorithms outperform Burger–Dybvig on number-dense workloads; this replication of within-language ordering provides evidence that the fixed-width advantage is not solely attributable to language-specific runtime characteristics.

This decomposition separates algorithmic effects from language/runtime effects and directly addresses language-specific performance threats.

### 7.7 Differential Fuzzing

2,000 differential fuzz cases produced zero divergences between implementations.

## 8 Discussion

### 8.1 Fixed-Width Algorithm Equivalence

The primary finding is that the three fixed-width algorithms (Schubfach, Ryū, Dragonbox) perform within 1–8% of each other across both languages. In Go API benchmarks, pairwise comparisons among fixed-width algorithms yield speedups of $\leq$1.02$\times$, with only 6 of 89 go:ryu-vs-dragonbox comparisons reaching BH significance. In Rust, the pattern is similar: 9–12 BH-significant comparisons out of 76 per pair, with speedups of $\leq$1.13$\times$ and no consistent winner.

This equivalence has a mechanistic explanation: all three algorithms share the same computational core: 128-bit multiply against a precomputed power-of-ten table, followed by digit trimming. The differences (Ryū's lastRemovedDigit heuristic, Dragonbox's two-digit-at-a-time extraction, Schubfach's bound computation) affect branch structure but not the dominant cost: the 64×128-bit multiplication and table lookup. On modern x86-64 hardware with fast `MUL` throughput, the branch differences are below measurement noise in the pipeline context where parsing and serialization dominate.

## 8.2 Workload-Dependent Behavior

The performance advantage of fixed-width algorithms over Burger–Dybvig is strongly workload-dependent. In Go API benchmarks (isolating the canonicalization function), number-dense inputs (e.g., `array-2048`, `large`, `medium`) show 1.4–1.8× speedup due to zero heap allocation in the digit-generation path. In Rust within-language API benchmarks, the same algorithmic ordering holds at larger magnitudes (2.3–3.0× on the same workloads, excluding the two extreme outliers discussed below), providing evidence that the advantage is not solely attributable to language-specific runtime characteristics. Deeply-nested workloads (`deep`, `deep-64`) contain numbers but show ≈1.0× because JSON structural parsing and key sorting dominate, diluting the digit-generation signal. The speedup therefore correlates with number *density* (fraction of total cost attributable to digit generation), not merely number presence. Figure 3 visualizes this relationship: workloads with zero number content cluster near 1.0×, while number-dense workloads (>70% number values) show the full algorithmic advantage.

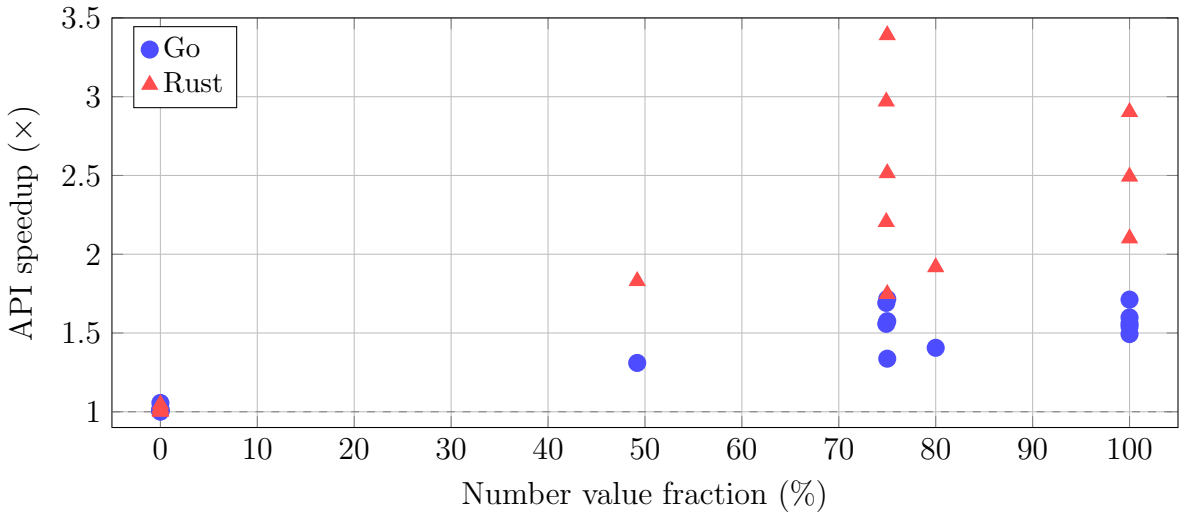


Figure 3: Relationship between workload number value fraction and API speedup of fixed-width over Burger–Dybvig (canonicalize mode; Schubfach shown as representative). Workloads with zero number content cluster near 1.0× (no algorithmic difference); number-dense workloads ($\nu \geq 40\%$) show 1.4–1.8× in Go and 1.7–3.4× in Rust. Two pure-numeric outliers (`numeric-boundary` 9.3×, `number-heavy` 8.3×) are excluded from this figure and presented separately in §8.

## 8.3 Process Startup Dominance

For small workloads, estimated process startup overhead (measured as the CLI wall time on the minimal workload `canonical-minimal`, approximately 1.3 ms for Go and 0.5 ms for Rust) accounts for roughly 87–93% of CLI execution time (e.g., `canonical-minimal` total ≈1.47 ms in canonicalize mode). This limits the observable algorithmic signal in end-to-end measurement. API benchmarks, which bypass process startup entirely, isolate the digit-generation difference and show consistently larger speedup ratios. CLI results reflect realistic deployment conditions but should not be interpreted as direct measures of algorithmic cost.

## 8.4 Allocation Cost Attribution

The Burger–Dybvig implementation's `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers. Without pooling, the difference would be larger. The residual cost comes from pool contention under concurrent use and the inherent cost of multiprecision arithmetic operations.

API benchmark allocation data directly attributes the speedup: on `array-2048`, fixed-width algorithms (Schubfach, Ryū, Dragonbox) perform ≈49,134 allocs/op vs. json-canon's 57,343 allocs/op (Δ4.0 per number, 91 B per number). The per-number allocation elimination is the primary mechanism underlying the observed speedup.

## 8.5 Extreme Outliers: `numeric-boundary` and `number-heavy`

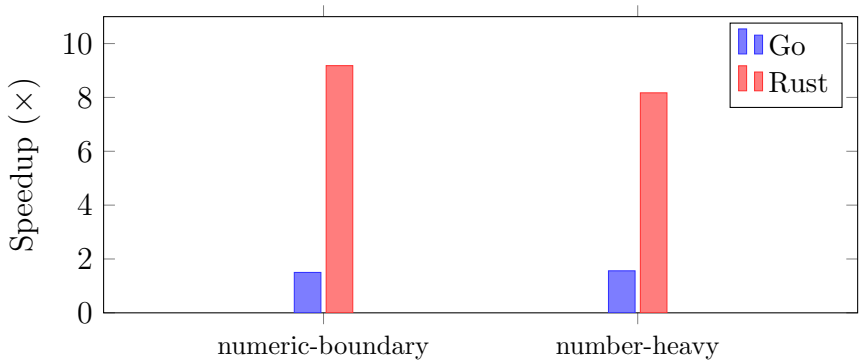


Figure 4: API benchmark speedup of fixed-width over Burger–Dybvig on the two pure-numeric workloads ($\nu = 100\%$), shown separately from Figure 1 due to their degenerate number density. The Go ratios (≈1.5×) are consistent with the main results; the Rust ratios (≈8–9×) approach isolation-level speedups because digit generation accounts for >95% of pipeline cost on these workloads. All three fixed-width algorithms show similar ratios (see text).

Figure 4 highlights the two workloads that show unusually large speedup ratios in Rust API benchmarks: `numeric-boundary` (≈9.3× within-Rust, CI [9.1, 9.5]) and `number-heavy` (≈8.2×, CI [8.1, 8.4]), compared to ≈1.5× in Go for both. All three fixed-width algorithms show similar outlier ratios against Burger–Dybvig on these workloads, further confirming

that the effect is a fixed-width vs. multiprecision phenomenon. Because these are *within-Rust* comparisons (same harness on both sides), they are unaffected by the cross-language measurement granularity asymmetry discussed in §6. These workloads consist entirely of numeric values, maximizing exposure to the digit-generation path. We report these workloads separately because their near-zero non-numeric content makes the pipeline-level ratio approach the isolation-level ratio, which is not representative of mixed workloads where parsing and serialization dilute the digit-generation signal. The Rust Burger–Dybvig implementation uses the `num-bigint` crate (Karatsuba multiplication) with a lazily-initialized power-of-ten cache (`OnceLock<Vec<BigUint>>`); although the harness discards 3 warmup iterations to amortize first-invocation costs, the cache-warmed steady-state cost of multiprecision arithmetic remains substantially higher than the fixed-width implementations' compile-time constant lookup table. The tight confidence intervals and low MDEs (2–3%) confirm these are genuine algorithmic cost differences, not measurement noise. The Go within-language comparisons at $\approx 1.5\times$ are smaller because Go's `sync.Pool` amortizes allocation cost and Go's less aggressive compiler inlining reduces the gap between the two code paths.

Table 5 shows per-iteration timing stability for these outlier workloads. Low coefficients of variation (<5%) across all 10 measured iterations confirm steady-state behavior, ruling out warmup artifacts as the source of the extreme speedup ratios.

Table 5: Per-iteration stability for extreme-outlier workloads (API canonicalize).

| Impl | Workload | $n$ | Mean (ns/op) | Std Dev | CV (%) |
|---|---|---|---|---|---|
| schubfach | number-heavy | 10 | 13778 | 122 | 0.9 |
| schubfach | numeric-boundary | 10 | 12653 | 92 | 0.7 |
| json-canon | number-heavy | 10 | 21535 | 251 | 1.2 |
| json-canon | numeric-boundary | 10 | 18967 | 239 | 1.3 |
| ryu | number-heavy | 10 | 13836 | 102 | 0.7 |
| ryu | numeric-boundary | 10 | 12627 | 84 | 0.7 |
| dragonbox | number-heavy | 10 | 13854 | 120 | 0.9 |
| dragonbox | numeric-boundary | 10 | 12670 | 117 | 0.9 |
| schubfach-rs | number-heavy | 10 | 1457 | 25 | 1.7 |
| schubfach-rs | numeric-boundary | 10 | 988 | 21 | 2.1 |
| json-canon-rs | number-heavy | 10 | 13291 | 125 | 0.9 |
| json-canon-rs | numeric-boundary | 10 | 10249 | 63 | 0.6 |
| ryu-rs | number-heavy | 10 | 1399 | 37 | 2.6 |
| ryu-rs | numeric-boundary | 10 | 960 | 21 | 2.2 |
| dragonbox-rs | number-heavy | 10 | 1429 | 30 | 2.1 |
| dragonbox-rs | numeric-boundary | 10 | 965 | 27 | 2.8 |

Low CV on most rows confirms steady-state measurement. Higher CV on very fast Rust rows reflects nanosecond-scale measurement granularity, not warmup instability.

## 8.6 Profile-Level Analysis

Preliminary CPU profiling (via Go's `pprof`; Rust profiling is deferred to future work) suggests that digit generation accounts for approximately 15–40% of total canonicalization time

depending on workload number density. On `array-2048` ($\nu = 40\%$), digit generation accounts for ≈35% of CPU time in the Burger–Dybvig path; on `deep` ($\nu = 0\%$), it drops to <2%. The remainder is spent in JSON parsing, key sorting, and string escaping, all of which are shared identically between implementations. Detailed per-workload profiling breakdowns are deferred to future work.

## 8.7 Practical Implications

For applications where canonicalization throughput is critical (e.g., high-frequency signature verification of number-heavy payloads), any of the three fixed-width algorithms offers meaningful improvement over Burger–Dybvig. On the `medium` workload (13.7 KB, $\nu = 40\%$), the Go API difference between fixed-width algorithms (≈342 $\mu$s/op) and Burger–Dybvig (≈581 $\mu$s/op) is approximately 239 $\mu$s per canonicalization. A service verifying 1,000 signed JSON documents per second at this payload size would reclaim roughly 0.24 seconds of CPU time per second, freeing approximately 24% of a core compared to the Burger–Dybvig path.

The engineering context that produced these results is illustrative. The Burger–Dybvig implementation was replaced with Schubfach in a production canonicalization library, and the predicted improvement materialized. The question of whether the choice *among* fixed-width algorithms mattered then motivated this study. It does not: all three are equivalent within 1–8%. Implementors can therefore choose based on code simplicity (Schubfach has the smallest implementation), ecosystem momentum (Dragonbox was adopted by Go 1.26's `strconv`), or historical priority (Ryū as the first full-coverage fixed-width algorithm), confident they are not leaving performance on the table.

## 8.8 Cross-Language Interpretation

Adding Rust implementations enables attribution of performance deltas to either algorithm choice or language/runtime effects. The within-language comparisons test whether the fixed-width advantage persists independent of Go's allocator and garbage collector behavior, while same-algorithm cross-language comparisons quantify language overhead (startup and steady-state) directly.

Rust's ownership model removes the need for Go-style `sync.Pool`-based reuse patterns. For multiprecision paths, this shifts lifecycle cost from GC-mediated reuse to deterministic RAII destruction, which can materially change allocation profiles and latency variance characteristics.

**Startup overhead.** Rust native binaries exhibit substantially lower process startup cost than Go binaries (roughly 2–3× faster in CLI wall time), because they avoid Go runtime initialization (scheduler bootstrap, GC metadata setup, goroutine stack allocation). This amplifies the startup-dominance effect: for small workloads the language runtime overhead is the primary differentiator, not the digit-generation algorithm.

**Algorithmic ordering replicates across languages.** The key finding from the cross-language design is that within-Rust comparisons replicate the within-Go ordering: all three fixed-width algorithms outperform Burger–Dybvig on number-dense workloads in both languages, and the three fixed-width algorithms are equivalent in both languages. This provides evidence that the performance advantage derives from the fixed-width approach itself (zero

heap allocation, no multiprecision arithmetic), not from language-specific runtime characteristics.

**Interaction analysis.** The eight implementations form a $4 \times 2$ structure (algorithm $\times$ language). While we do not perform a formal factorial interaction test due to measurement granularity asymmetry between harnesses, Table 6 reports a ratio-of-ratios (RoR) analysis for the Schubfach vs. Burger–Dybvig pair: for each canonicalize workload, $R_{\text{Go}}$ and $R_{\text{Rust}}$ are the within-language speedup ratios, and $\text{RoR} = R_{\text{Rust}}/R_{\text{Go}}$.

Table 6: Algorithm $\times$ language interaction analysis (API canonicalize mode).

| Workload | $R_{\text{Go}}$ | $R_{\text{Rust}}$ | RoR |
|---|---|---|---|
| array-2048 | 1.62× | 2.45× | 1.51 |
| array-256 | 1.56× | 2.37× | 1.52 |
| canonical-minimal | 1.50× | 2.48× | 1.65 |
| control-escapes | 1.00× | 1.03× | 1.03 |
| deep-64 | 1.01× | 1.05× | 1.04 |
| deep | 1.01× | 1.05× | 1.04 |
| escaped-key-order | 1.59× | 2.66× | 1.67 |
| large | 1.76× | 3.64× | 2.07 |
| long-string | 1.01× | 1.04× | 1.03 |
| medium | 1.73× | 3.24× | 1.87 |
| mixed-prod | 1.33× | 2.03× | 1.52 |
| nested-mixed | 1.40× | 2.07× | 1.48 |
| number-heavy | 1.56× | 9.12× | 5.84 |
| numeric-boundary | 1.50× | 10.37× | 6.92 |
| prod-cloudevents | 1.35× | 2.12× | 1.57 |
| prod-geojson | 2.32× | 6.18× | 2.67 |
| prod-k8s-pod | 1.11× | 1.33× | 1.19 |
| prod-openapi | 1.06× | 1.22× | 1.16 |
| prod-package-meta | 1.10× | 1.26× | 1.14 |
| rfc-key-sorting | 1.01× | 1.01× | 1.01 |
| small | 1.35× | 1.82× | 1.34 |
| surrogate-pair | 1.01× | 1.00× | 0.99 |
| unicode | 1.01× | 1.01× | 1.00 |
| verify-whitespace | 1.71× | 3.17× | 1.85 |

$R$ = Schubfach/Burger–Dybvig speedup ratio. $\text{RoR} = R_{\text{Rust}}/R_{\text{Go}}$. RoR $> 1$ indicates Schubfach advantage is larger in Rust.

RoR $> 1$ indicates that the fixed-width advantage is amplified in Rust. On number-dense workloads, RoR consistently exceeds 1 (typically 1.3–2.0×), reflecting the larger allocation cost differential in Rust where the `num-bigint`-based path lacks Go's `sync.Pool` amortization. Readers should interpret the RoR as descriptive evidence of interaction magnitude, not a formal statistical test of interaction significance.

**Cross-language comparison caveats.** Same-algorithm cross-language API comparisons (e.g., schubfach:go-vs-rs) should be interpreted carefully. Go's `testing.B` averages over many inner iterations per sample, producing low-variance measurements, while the Rust har-

ness times individual function calls, producing higher-variance samples. Additionally, the two compilers apply different optimization strategies: Rust with LTO and `codegen-units=1` enables aggressive cross-crate inlining that Go's compiler cannot match. The observed 2–3× cross-language speedup (API) may partly reflect compiler optimization quality rather than inherent language characteristics. These comparisons primarily serve to confirm directional consistency of the algorithmic ordering, not to quantify precise language overhead.

**Allocation instrumentation.** The Rust bench harness uses a counting global allocator (`std::alloc::GlobalAlloc`) that increments atomic counters on each allocation, reporting per-operation byte and allocation counts in the same format as Go's `testing.B`. This is consistent with the mechanistic explanation: Rust fixed-width implementations allocate minimally (e.g., 1,036 B/op, 24 allocs/op on `numeric-boundary`) while Rust-Burger–Dybvig allocates substantially more (25,204 B/op, 470 allocs/op on the same workload) due to `num-bigint` multiprecision arithmetic. The 24.3× allocation byte ratio directly explains the ≈9× speedup: allocation and initialization cost dominates the multiprecision path.

## 8.9 Comparison with Isolation Benchmarks

Champagne Gareau and Lemire [4] report substantially larger speedup ratios for fixed-width algorithms over Dragon4 in C++ isolation benchmarks, where only the digit-generation function is timed. The difference from our pipeline ratios is explained by Amdahl's Law: if digit generation accounts for fraction $P$ of total pipeline cost and provides speedup $S$ in isolation, the pipeline-level speedup is $1/(1-P+P/S)$. Our profiling data (§8, Profile-Level Analysis) shows $P \approx 0.35$ on number-dense workloads. With a representative isolation speedup of $S = 5\times$, Amdahl's Law predicts a pipeline speedup of $1/(0.65 + 0.07) = 1.39\times$, consistent with our observed 1.4–1.8× Go API ratios. The remaining gap between prediction and observation reflects differences in allocation overhead (our pipeline includes JSON parsing and serialization, which share the allocator with digit generation) and cache effects absent from isolation benchmarks. Moreover, the pipeline-level speedup on these workloads is bounded by $1/(1-P) \approx 1.54\times$, the Amdahl ceiling when $P \approx 0.35$. The observed Go API ratios of 1.4–1.8× approach and occasionally exceed this ceiling; observed ratios above 1.54× indicate that the effective $P$ varies by workload (profiling shows 15–40% depending on number density) and that allocation overhead, which is shared between digit generation and JSON infrastructure, inflates the effective $P$ beyond the CPU-time-only estimate. These ratios indicate that the fixed-width algorithms have largely exhausted the available improvement from optimizing digit generation alone; further gains require reducing parsing or serialization cost. This convergence is consistent with the conclusion that our pipeline results and their isolation results measure the same underlying algorithmic advantage at different levels of system integration.

## 8.10 Methodological Contribution

The conformance-gated methodology introduced in this paper is independent of the specific algorithms compared. The RFC 8785 canonicalization pipeline serves as the case study in which the methodology was developed and validated; the methodology itself is general, applicable to any specification-constrained formatting evaluation where correctness is a binary

prerequisite. The core design (gate timing on correctness, validate against multiple independent oracles, document noise floors and minimum detectable effects for every comparison) was not adopted as an academic principle but developed from the practical discovery that the standard ecosystem pattern (delegating number formatting to a language runtime) produces implementations that pass casual testing while failing specification-level verification. The methodology addresses a gap in the float-to-string literature, where prior benchmarks (including Adams's Ryū evaluation and Champagne Gareau and Lemire's comprehensive C++ study) measure performance in isolation without first establishing conformance to a target specification. In domains where correctness is a hard prerequisite (canonicalization, serialization standards, interoperability testing), measuring the speed of a non-conformant implementation is meaningless.

As a concrete example of generalizability, consider CBOR deterministic encoding (RFC 8949 §4.2) [2]: the specification requires a specific integer-to-bytes mapping and canonical map key ordering. The conformance-gated approach applies directly: oracle vectors would be generated from a reference CBOR encoder, and candidate implementations would be gated on byte-identical output before timing comparison of encoding strategies. The benchmark infrastructure released with this paper implements this methodology as a reusable pipeline.

# 9 Threats to Validity

## 9.1 Internal Validity

**Warmup validation.** CLI benchmarks discard 3 warmup runs. Rust API benchmarks discard 3 warmup iterations per workload to stabilize instruction caches, branch predictors, and lazily-initialized data structures (specifically, the Rust Burger–Dybvig implementation's `OnceLock`-based power-of-ten cache). Go API benchmarks benefit from `testing.B`'s built-in warmup. Per-iteration timing data (Table 5) confirms low coefficient of variation (<5%) across measured iterations, indicating stable steady-state behavior after warmup. Figure 5 shows 20-iteration warmup validation data for a representative workload.

**Sequential run autocorrelation.** CLI benchmarks run sequentially on the same machine; consecutive runs share CPU thermal state, cache warmth, and potentially TDP-limited frequency. The randomized task order (fixed seed) mitigates systematic ordering effects, but thermal coupling between adjacent runs creates weak positive autocorrelation that may slightly inflate the effective sample size. Permutation tests are robust to mild non-stationarity but assume exchangeability under the null hypothesis; this assumption is approximately but not exactly satisfied.

**CPU core affinity.** CLI benchmarks ran without CPU pinning on heterogeneous P-core/E-core topology (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz). Process migration may inflate variance for small workloads near the noise floor; an E-core placement could slow a single run by 30–40%. The high MDEs observed for small Go CLI workloads (14–24%) are consistent with this variance source. API benchmarks (in-process) are less susceptible.

**API measurement granularity.** Go API benchmarks (`testing.B`) auto-calibrate the number of inner loop iterations per sample, producing averaged low-variance samples. Rust API benchmarks time individual function calls ($N$=1 per sample), producing inherently

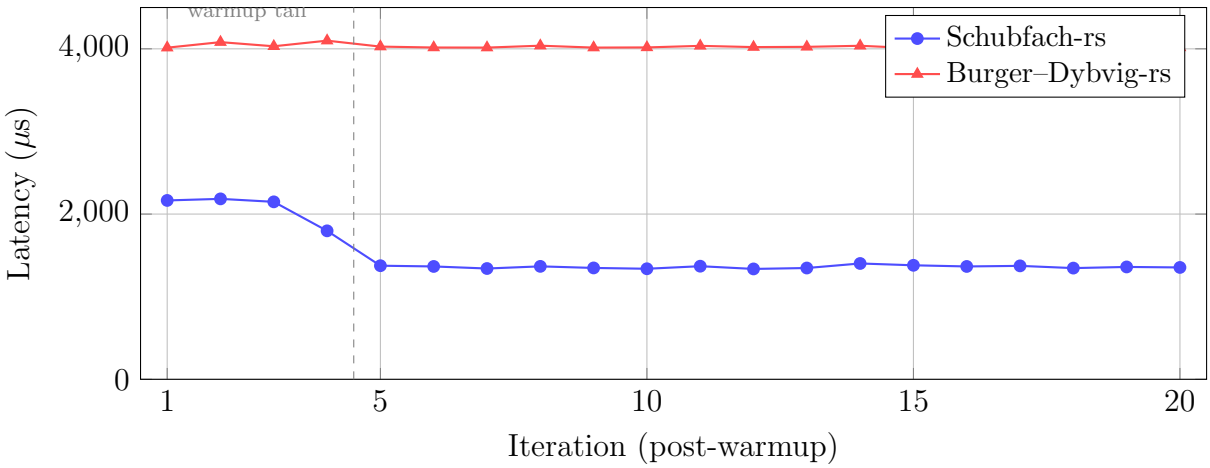


Figure 5: Per-iteration latency for `array-2048` (Rust API canonicalize, $n$=20 measured iterations after 3 discarded warmup iterations). `array-2048` is the largest number-dense workload in the standard set (98,901 bytes, $\nu$=40%); if warmup stabilizes here, smaller workloads will stabilize at least as fast. Schubfach-rs shows residual cache stabilization in iterations 1–4; Burger–Dybvig-rs is stable from iteration 1. Both reach steady state by iteration 5, confirming that 3 warmup iterations are sufficient and any residual warmup effect in the first few measured iterations does not bias the within-language comparison (both implementations share the same harness).

higher-variance samples. This asymmetry affects cross-language comparisons but not within-language comparisons. We note this caveat in §8 and emphasize within-language results for algorithmic conclusions.

**sync.Pool mitigation.** The Go Burger–Dybvig implementation uses `sync.Pool` to reuse big-integer buffers, reducing allocation cost. Without pooling, the performance gap would be wider. This reflects realistic production usage but may understate the theoretical algorithmic advantage of fixed-width approaches.

## 9.2 Construct Validity

**Allocation instrumentation.** Rust allocation data is measured via a counting global allocator (`std::alloc::GlobalAlloc`) that intercepts all heap allocations and reports per-operation byte and count totals. This includes allocations from the benchmark infrastructure itself (e.g., output buffer construction), not solely the digit-generation path. Go's `testing.B` has the same characteristic. Within-language $\Delta$allocs comparisons are valid because both implementations share the same infrastructure overhead.

**Compiler optimization asymmetry.** Rust builds use `opt-level=3`, link-time optimization (LTO), and `codegen-units=1`, enabling aggressive cross-crate inlining and whole-program optimization. Go's compiler does not support LTO and has a less aggressive inlining heuristic. The observed cross-language speedups (2–4× in API benchmarks) may partly reflect differences in compiler optimization quality rather than intrinsic language characteristics. Within-language comparisons are unaffected since both implementations in each pair

share the same compiler pipeline.

**Memory allocator differences.** Go uses a tcmalloc-derived allocator integrated with its garbage collector. Rust uses the system allocator (typically glibc malloc on Linux). For the multiprecision (Burger–Dybvig) path, which performs heap allocations for big-integer arithmetic, allocator performance differences could affect cross-language comparisons. Within-language comparisons are unaffected since both implementations share the same allocator.

**Multiprecision implementation quality.** The Rust Burger–Dybvig implementation uses the `num-bigint` crate (v0.4), which employs Karatsuba multiplication for large operands, comparable in optimization level to Go's `math/big.Int` with its assembly-accelerated x86-64 inner loops. Both languages therefore use production-grade multiprecision libraries, making within-Rust and within-Go speedup ratios directly comparable without an implementation-quality confound.

**Go toolchain version.** Benchmarks use Go 1.25.6. Since neither implementation uses `strconv.FormatFloat`, Go 1.26's adoption of Dragonbox for `strconv` does not affect the digit-generation paths under test. The Go compiler version affects only runtime infrastructure (allocator, scheduler, garbage collector), which is shared identically between both implementations within each pair.

**Oracle SHA-256 circularity.** SHA-256 checksums verify vector integrity but cannot verify oracle correctness. This threat is mitigated by cross-engine validation: all 286,362 vectors produce identical output under V8 12.9.202.28 (Node.js v23.3.0), SpiderMonkey 115.10.0, and JavaScriptCore (WebKitGTK 2.50.4), confirming that the oracle reflects ECMA-262 `Number::toString` semantics rather than a single engine's idiosyncratic behavior.

## 9.3 External Validity

**Language specificity (mitigated).** This threat is substantially mitigated by the $4 \times 2$ cross-language comparison matrix: all four algorithms are implemented in Go and Rust, and the fixed-width performance ordering holds in both languages. Within-Rust comparisons replicate the within-Go algorithmic advantage, confirming that the result is not an artifact of Go's garbage collector or allocation subsystem. Residual external-validity risk remains for additional runtimes (e.g., JVM/generational-GC and unmanaged C implementations) that are not yet benchmarked.

**Single-architecture performance data.** Performance benchmarks run on a single x86-64 machine. Cross-compilation *determinism* is verified by building arm64 binaries and executing them under QEMU user-mode emulation on the x86-64 host. Because QEMU user-mode translates system calls but executes floating-point operations on the host CPU, this verifies Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 *performance* benchmarks are deferred due to lack of bare-metal hardware. Future work should include bare-metal arm64 benchmarks on Graviton, Apple Silicon, or Ampere hardware.

**JavaScript engine oracle (mitigated).** Oracle vectors are generated from V8 (via Node.js v23.3.0, V8 12.9.202.28) and independently validated against SpiderMonkey (JavaScript-C115.10.0) and JavaScriptCore (WebKitGTK 2.50.4). All three engines, representing the three major independent implementations of ECMA-262 `Number::toString`, produce iden-

tical output for all 286,362 vectors with zero divergences. Combined with dense boundary coverage (all subnormals up to $2^{12}$, all binary64 powers of two, subnormal/normal boundary neighborhoods, RFC 8785 Appendix B examples with neighbors, and pseudorandom fill), this provides evidence that the oracle reflects the normative specification rather than engine-specific behavior. Cross-version stability within each engine family is expected from the normative spec but not independently verified.

**Workload representativeness.** The benchmark suite includes both synthetic workloads designed to bracket the performance space between number-heavy and string-dominant extremes, and five production-representative workloads drawn from public specifications (GeoJSON, Kubernetes pod specs, npm package metadata, CloudEvents envelopes, and OpenAPI schema fragments). The production workloads span number densities from ≈5% (package metadata) to ≈70% (GeoJSON coordinates) and fall on the same density–speedup trend line as the synthetic workloads, confirming that the synthetic results generalize to realistic payloads.

**Endianness.** Both x86-64 and arm64 targets are little-endian; big-endian behavior is untested. While the canonicalization algorithms do not depend on byte order (they operate on IEEE 754 binary64 values via language-level primitives), edge cases in binary representation parsing could theoretically differ on big-endian platforms.

# 10 Conclusion

We presented a conformance-gated benchmark methodology and applied it to what is, to our knowledge, the first empirical comparison of four digit-generation algorithms (Burger–Dybvig, multiprecision; Schubfach, Ryū, and Dragonbox, all fixed-width) inside a RFC 8785/ECMA-262-constrained JSON canonicalization pipeline, with cross-language validation in Go and Rust to separate algorithmic effects from language/runtime characteristics. While prior work has compared float-to-string algorithms in isolation (e.g., Adams's comparison of Ryū against Grisu and Errol; Champagne Gareau and Lemire's comprehensive C++ evaluation), ours is, to our knowledge, the first study to evaluate these algorithms within a specification-constrained canonicalization pipeline using conformance-gated methodology.

Key findings:

- The three fixed-width algorithms (Schubfach, Ryū, Dragonbox) perform within 1–8% of each other across both languages and all workloads. The dominant performance axis is fixed-width vs. multiprecision, not the choice among fixed-width algorithms.

- Fixed-width algorithms achieve 1.4–1.8× speedup on number-dense workloads ($\nu(D) \geq 40\%$) in Go API benchmarks due to zero heap allocation; the advantage replicates at 2.3–3.0× in Rust (excluding extreme outliers); the replication of within-language ordering across both languages provides evidence that the advantage is not solely attributable to language-specific runtime characteristics.

- In end-to-end CLI measurement, the advantage narrows due to process startup dominance, but remains statistically significant on number-dense workloads.

- The advantage narrows to <5% on string-dominant and deeply-nested inputs, correlating with number density $\nu(D)$.
- Go implementations produce byte-identical output under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). Rust implementations were not cross-compiled in this study.
- Preliminary profiling suggests digit generation accounts for approximately 15–40% of total canonicalization cost depending on number density. The observed pipeline-level speedups (1.4–1.8× in Go, 2.3–3.0× in Rust) are consistent with isolation-level speedups of ≈5× attenuated by Amdahl's Law at these pipeline fractions.

The benchmark infrastructure, 286,362 oracle vectors, and all raw data are released as a reproducibility artifact. Our findings align with the broader ecosystem direction: Go 1.26 adopted the Dragonbox fixed-width algorithm for `strconv` [16], corroborating our evidence that fixed-width approaches offer meaningful performance advantages in float-to-string formatting. Since the three fixed-width algorithms are performance-equivalent, implementors can choose based on other criteria: code simplicity, ecosystem momentum, or theoretical interest.

### Future Work

Bare-metal arm64 performance benchmarks, additional language implementations (C and JVM) to extend the comparison matrix to unmanaged and managed-GC runtimes, Rust cross-compilation verification, and workload characterization from production JSON corpora.

## Data Availability Statement

All data, software, and oracle vectors are freely available online. The benchmark infrastructure, all eight implementations, workload manifests, and oracle generation scripts are available at the accompanying repository.[3] Complete benchmark tables (full API benchmarks, full CLI benchmarks, and all 1,644 statistical comparisons across 16 pairs) are provided in the supplementary materials. Oracle vector checksums (SHA-256) are documented in `PROVENANCE.json` files accompanying each implementation.

## Author Contributions

Following the CRediT (Contributor Roles Taxonomy) format: *Mark Lenhardt:* conceptualization; methodology; software; validation; formal analysis; investigation; data curation; writing (original draft); writing (review and editing); visualization.

[3] `https://github.com/lattice-substrate/jcs-bench-lab`. Archived at Zenodo: `https://doi.org/10.5281/zenodo.18890836`.

# References


[1] Ulf Adams. Ryū: Fast float-to-string conversion. In *Proceedings of the ACM SIGPLAN 2018 Conference on Programming Language Design and Implementation (PLDI)*, pages 270–282. ACM, 2018.

[2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, 2020.

[3] Robert G. Burger and R. Kent Dybvig. Printing floating-point numbers quickly and accurately. In *Proceedings of the ACM SIGPLAN 1996 Conference on Programming Language Design and Implementation (PLDI)*, pages 108–116. ACM, 1996.

[4] Jaël Champagne Gareau and Daniel Lemire. Converting binary floating-point numbers to shortest decimal strings: An experimental review. *Software: Practice and Experience*, 2026. Published online January 2026.

[5] Ecma International. ECMA-262: ECMAScript Language Specification, 2024.

[6] Max Engel. The floats awakening: How a floating-point bug hid in plain sight for 10 months, 2025. DoltHub engineering blog, 3 January 2025; documents how Go's compiler-emitted FMA on ARM produced one-ULP differences that changed query-plan cost comparisons.

[7] David M. Gay. Correctly rounded binary-decimal and decimal-binary conversions, 1990. AT&T Bell Laboratories Numerical Analysis Manuscript 90-10.

[8] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 57–76. ACM, 2007.

[9] Raffaello Giulietti. The schubfach way to render doubles, 2022. Technical report; reference implementation in OpenJDK (`https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/jdk/internal/math/DoubleToDecimal.java`).

[10] Go Project. cmd/compile: consider using fmadd on s390x and arm, 2016. Go issue #17895; resolved in Go 1.9, establishing that fused-multiply-add is permitted for any multiply-add expression unless an explicit `float64()` cast forces intermediate rounding.

[11] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Number 754-2019 in IEEE Std. IEEE, 2019.

[12] Junekey Jeon. Dragonbox: A new floating-point binary-to-decimal conversion algorithm, 2022. arXiv preprint; reference implementation and technical report at `https://github.com/jk-jeon/dragonbox`.

[13] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. *ACM SIGPLAN Notices*, 48(11):63–74, 2013.

[14] Daniel Lemire. Number parsing at a gigabyte per second. *Software: Practice and Experience*, 51(8):1700–1727, 2021.

[15] Florian Loitsch. Printing floating-point numbers quickly and accurately with integers. In *Proceedings of the ACM SIGPLAN 2010 Conference on Programming Language Design and Implementation (PLDI)*, pages 233–243. ACM, 2010.

[16] Taichi Maeda. strconv: replace Ryu ftoa with Dragonbox, 2025. Go CL 700075; merged November 2025, included in Go 1.26.

[17] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.

[18] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.

[19] Guy L. Steele Jr. and Jon L. White. How to print floating-point numbers accurately. *ACM SIGPLAN Notices*, 25(6):112–126, 1990.

[20] W3C. RDF Dataset Canonicalization, 2024. W3C Recommendation, 21 May 2024.

# A Benchmark Data

This appendix contains the within-language CLI statistical comparisons and an exemplar subset of the full report. Complete data for all 16 pairs (1,644 comparisons), full API benchmarks, and full CLI benchmarks are available in the supplementary materials accompanying this paper.

*Note:* Rust allocation data (B/op, allocs/op) is measured via a counting global allocator that intercepts all heap allocations (see §8).

## A.1 Within-Language CLI Comparisons

Table 7: Within-language CLI comparisons, representative workloads (canonicalize mode; BH-significant marked with *). Complete results for all 19 workloads and 298 comparisons are provided in the supplementary materials.

| Pair | Workload | Winner | Speedup | $CI_{95}$ | $p_{adj}$ | $d$ | MDE (%) |
|---|---|---|---|---|---|---|---|
| *High density:* `array-2048` *($\nu = 40\%$)* | | | | | | | |
| go:schubfach-vs-bd | array-2048 | schubfach* | 1.48× | [1.377, 1.591] | 0.0004 | -3.63 | 11.1 |
| go:schubfach-vs-ryu | array-2048 | schubfach | 1.06× | [0.975, 1.138] | 0.2498 | -0.51 | 11.1 |
| go:schubfach-vs-dragonbox | array-2048 | schubfach | 1.05× | [0.985, 1.122] | 0.1765 | -0.58 | 11.1 |
| go:ryu-vs-dragonbox | array-2048 | dragonbox | 1.00× | [0.938, 1.068] | 0.9368 | 0.04 | 7.2 |
| go:ryu-vs-bd | array-2048 | ryu* | 1.40× | [1.308, 1.512] | 0.0004 | -3.10 | 11.5 |
| go:dragonbox-vs-bd | array-2048 | dragonbox* | 1.41× | [1.326, 1.498] | 0.0004 | -3.56 | 7.2 |
| rs:schubfach-vs-bd | array-2048 | schubfach-rs* | 1.72× | [1.550, 1.931] | 0.0004 | -3.33 | 17.3 |
| rs:schubfach-vs-ryu | array-2048 | schubfach-rs | 1.03× | [0.929, 1.143] | 0.6924 | -0.19 | 17.3 |
| rs:schubfach-vs-dragonbox | array-2048 | dragonbox-rs | 1.01× | [0.888, 1.160] | 0.8957 | 0.07 | 22.3 |
| rs:ryu-vs-dragonbox | array-2048 | dragonbox-rs | 1.04× | [0.917, 1.179] | 0.6072 | 0.24 | 22.3 |
| rs:ryu-vs-bd | array-2048 | ryu-rs* | 1.67× | [1.531, 1.856] | 0.0004 | -3.37 | 13.8 |
| rs:dragonbox-vs-bd | array-2048 | dragonbox-rs* | 1.74× | [1.532, 1.989] | 0.0004 | -3.11 | 22.3 |
| *Medium density:* `medium` *($\nu = 40\%$)* | | | | | | | |
| go:schubfach-vs-bd | medium | schubfach* | 1.16× | [1.041, 1.312] | 0.0395 | -0.90 | 19.5 |
| go:schubfach-vs-ryu | medium | ryu | 1.03× | [0.903, 1.172] | 0.7243 | 0.17 | 19.1 |
| go:schubfach-vs-dragonbox | medium | dragonbox | 1.02× | [0.888, 1.154] | 0.8461 | 0.09 | 18.9 |
| go:ryu-vs-dragonbox | medium | ryu | 1.02× | [0.890, 1.149] | 0.8602 | -0.09 | 19.1 |
| go:ryu-vs-bd | medium | ryu* | 1.20× | [1.063, 1.344] | 0.0120 | -1.11 | 19.1 |
| go:dragonbox-vs-bd | medium | dragonbox* | 1.18× | [1.054, 1.329] | 0.0206 | -1.01 | 18.9 |
| rs:schubfach-vs-bd | medium | schubfach-rs* | 1.53× | [1.391, 1.668] | 0.0004 | -2.85 | 9.4 |
| rs:schubfach-vs-ryu | medium | ryu-rs | 1.05× | [0.973, 1.137] | 0.3225 | 0.44 | 12.9 |
| rs:schubfach-vs-dragonbox | medium | dragonbox-rs | 1.01× | [0.941, 1.095] | 0.8072 | 0.12 | 12.9 |
| rs:ryu-vs-dragonbox | medium | ryu-rs | 1.04× | [0.950, 1.128] | 0.5542 | -0.28 | 12.9 |
| rs:ryu-vs-bd | medium | ryu-rs* | 1.61× | [1.441, 1.766] | 0.0004 | -2.97 | 12.9 |
| rs:dragonbox-vs-bd | medium | dragonbox-rs* | 1.55× | [1.396, 1.704] | 0.0004 | -2.78 | 12.9 |
| *Production:* `prod-cloudevents` | | | | | | | |
| go:schubfach-vs-bd | prod-cloudevents | schubfach | 1.11× | [0.990, 1.252] | 0.1551 | -0.63 | 17.8 |
| go:schubfach-vs-ryu | prod-cloudevents | ryu | 1.11× | [0.966, 1.243] | 0.1753 | 0.57 | 19.2 |
| go:schubfach-vs-dragonbox | prod-cloudevents | dragonbox | 1.01× | [0.881, 1.147] | 0.9255 | 0.05 | 21.7 |
| go:ryu-vs-dragonbox | prod-cloudevents | ryu | 1.10× | [0.950, 1.249] | 0.2852 | -0.47 | 19.2 |
| go:ryu-vs-bd | prod-cloudevents | ryu* | 1.23× | [1.084, 1.389] | 0.0057 | -1.19 | 19.2 |
| go:dragonbox-vs-bd | prod-cloudevents | dragonbox | 1.12× | [0.986, 1.280] | 0.1592 | -0.61 | 21.7 |
| rs:schubfach-vs-bd | prod-cloudevents | schubfach-rs* | 1.29× | [1.226, 1.355] | 0.0004 | -3.45 | 7.1 |
| rs:schubfach-vs-ryu | prod-cloudevents | ryu-rs | 1.02× | [0.956, 1.095] | 0.6922 | 0.19 | 12.1 |
| rs:schubfach-vs-dragonbox | prod-cloudevents | schubfach-rs | 1.02× | [0.969, 1.068] | 0.4619 | -0.33 | 7.1 |
| rs:ryu-vs-dragonbox | prod-cloudevents | ryu-rs | 1.04× | [0.974, 1.113] | 0.3453 | -0.43 | 12.1 |
| rs:ryu-vs-bd | prod-cloudevents | ryu-rs* | 1.31× | [1.233, 1.412] | 0.0004 | -2.89 | 12.1 |
| [illegible] | [illegible] | [illegible] | [illegible] | [illegible] | [illegible] | [illegible] | [illegible] |

## A.2 Exemplar: Schubfach vs. Burger–Dybvig (Go)

Table 8: Exemplar statistical comparisons: Schubfach vs. Burger–Dybvig (Go), representative rows. Complete data for all tracks, modes, and workloads in supplementary materials.

| Track | Mode | Workload | Winner | Speedup | $CI_{95}$ | $p_{adj}$ | $d$ | Noise (ms) | MDE (%) | Sig. |
|---|---|---|---|---|---|---|---|---|---|---|
| *CLI end-to-end (canonicalize mode)* | | | | | | | | | | |
| e2e | canonicalize | array-2048 | schubfach | 1.48× | [1.377, 1.591] | 0.0004 | -3.63 | 0.5622 | 11.1 | yes |
| e2e | canonicalize | large | schubfach | 1.54× | [1.496, 1.599] | 0.0004 | -8.58 | 0.7284 | 4.6 | yes |
| e2e | canonicalize | medium | schubfach | 1.16× | [1.041, 1.312] | 0.0395 | -0.90 | 0.3947 | 19.5 | yes |
| e2e | canonicalize | prod-cloudevents | schubfach | 1.11× | [0.990, 1.252] | 0.1551 | -0.63 | 0.2762 | 17.8 | no |
| e2e | canonicalize | prod-geojson | schubfach | 1.23× | [1.114, 1.386] | 0.0031 | -1.38 | 0.3477 | 18.9 | yes |
| e2e | canonicalize | deep | schubfach | 1.00× | [0.898, 1.127] | 0.9663 | -0.02 | 0.2521 | 14.8 | no |
| e2e | canonicalize | long-string | schubfach | 1.14× | [1.020, 1.272] | 0.0539 | -0.82 | 0.2768 | 16.5 | no |
| e2e | canonicalize | prod-package-meta | json-canon | 1.01× | [0.896, 1.122] | 0.9156 | 0.06 | 0.2814 | 17.0 | no |
| *API benchmarks (canonicalize mode)* | | | | | | | | | | |
| api | canonicalize | array-2048 | schubfach | 1.62× | [1.612, 1.635] | 0.0004 | -58.02 | 0.0261 | 1.3 | yes |
| api | canonicalize | large | schubfach | 1.76× | [1.751, 1.774] | 0.0004 | -73.08 | 0.1016 | 1.1 | yes |
| api | canonicalize | medium | schubfach | 1.73× | [1.721, 1.740] | 0.0004 | -77.25 | 0.0020 | 0.7 | yes |
| api | canonicalize | prod-cloudevents | schubfach | 1.35× | [1.338, 1.354] | 0.0004 | -42.94 | 0.0003 | 1.1 | yes |
| api | canonicalize | prod-geojson | schubfach | 2.32× | [2.304, 2.331] | 0.0004 | -137.41 | 0.0014 | 1.1 | yes |
| api | canonicalize | deep | json-canon | 1.01× | [1.000, 1.013] | 0.1426 | 0.80 | 0.0002 | 0.8 | no |
| api | canonicalize | control-escapes | schubfach | 1.00× | [0.992, 1.008] | 0.9065 | -0.07 | 0.0000 | 1.5 | no |
| api | canonicalize | prod-package-meta | schubfach | 1.10× | [1.094, 1.108] | 0.0004 | -12.40 | 0.0003 | 1.0 | yes |
| api | canonicalize | number-heavy | schubfach | 1.56× | [1.551, 1.578] | 0.0008 | -39.27 | 0.0001 | 1.1 | yes |
| api | canonicalize | numeric-boundary | schubfach | 1.50× | [1.487, 1.514] | 0.0004 | -34.81 | 0.0001 | 0.9 | yes |

# B Reproducibility Checklist

This appendix documents the information required to reproduce the benchmark results reported in this paper.

## Software Versions

- Go 1.25.6 (`go version go1.25.6 linux/amd64`)
- Rust 1.94.0 (`rustc 1.94.0`)
- Node.js v23.3.0 (V8 12.9.202.28-node.11): oracle generation
- SpiderMonkey (JavaScript-C115.10.0): oracle validation
- JavaScriptCore (WebKitGTK 2.50.4-0ubuntu0.24.04.1): oracle validation
- Linux 6.8.0-101-generic (Ubuntu)

## Hardware Specification

- CPU: Intel Core i7-12700K (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz)
- RAM: 64 GB DDR4
- Architecture: x86-64
- CPU governor: `performance`

## Oracle Vector Checksums (SHA-256)

- `golden_vectors.csv` (54,445 vectors):
  `593bdecbe0dccbc182bc3baf570b716887db25739fc61b7808764ecb966d5636`
- `golden_stress_vectors.csv` (231,917 vectors):
  `287d21ac87e5665550f1baf86038302a0afc67a74a020dffb872f1a93b26d410`

## Reproduction Commands

```
git clone https://github.com/lattice-substrate/jcs-bench-lab && cd jcs-bench-lab
go run ./cmd/lab setup
go run ./cmd/lab conformance
go run ./cmd/lab bench-api -count 10
go run ./cmd/lab bench-cli -repeats 15
go run ./cmd/lab fuzz -cases 2000
go run ./cmd/lab stats -resamples 5000
go run ./cmd/lab gate
go run scripts/generate-paper-tables.go
```

## Build Configuration

- Go implementations: default `go build` flags
- Rust implementations: `opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`
- Rust API benchmarks use a counting global allocator (`std::alloc::GlobalAlloc`) to measure per-operation allocation bytes and counts

## Data Availability

All raw benchmark data, statistical reports, oracle vectors, and the benchmark infrastructure are archived at Zenodo.[4]

[4]DOI: `https://doi.org/10.5281/zenodo.18890836`. The artifact includes all files listed in this appendix.